

A Mind Made of Data

The Science Behind Artificial Intelligence

About This Book

The era of artificial intelligence that seemed to be something out of science fiction novels is now here; it is already an integral part of learning, working, creating, and decision-making.

This book presents an introductory overview of the theory and technologies of artificial intelligence. You will learn how computers are taught through data analysis, how neural networks function, how modern artificial intelligence technologies produce texts and images, and how the systems which we use every day work inside.

While explaining what AI can do is not enough to fully grasp the technology, the primary purpose of this book is to provide insight into why and how it works and its boundaries.

This book is written for people who are just getting acquainted with AI technologies or those who have already tried working with it.

Copyright © 2026 Abdallah Mohamed
All rights reserved.

First Edition, **2026**

Published by **EYAIF, Egyptian Youth Artificial Intelligence Foundation.**

This book is an original educational work. No part of this publication may be reproduced, distributed, or transmitted without prior permission from the copyright holder, except where permitted by law.



About the Author

Abdallah Mohamed
Founder & Lead Mentor of **EYAIF**
Grade 12 | Sharkya STEM School
First Edition — 2026



Abdallah Mohamed is an Egyptian student studying in Sharkya STEM School, Grade 12, Senior 2027. His academic interests are primarily focused on Mathematics, Computer Science, and Artificial Intelligence.

Abdallah's academic interests are based on mathematics and computation, specifically the way in which mathematical concepts and computations can be used to build intelligent systems. His mathematics education has made him interested in concepts not covered in schools, and his interest in Computer Science has made him curious about algorithms and problem solving through computations, including basic principles of Artificial Intelligence.

Abdallah has been working on Artificial Intelligence via self-learning, research, and various projects. His interests include Machine Learning, Neural Networks, intelligent systems, and the use of Artificial Intelligence for solving issues related to education, science, and disaster situations.

Abdallah is specifically interested in the conceptual background of AI rather than seeing it as a tool box of techniques. His interests include ways that machines learn, how models detect patterns, algorithms making predictions, and building increasingly complicated systems from the basic mathematics and computation.

Abdallah is a learner who learns by doing experiments, conducting research, and self-study. His passion lies in the ability to dissect complex concepts and technologies in science and engineering and apply theories in practical life.

This way of learning is seen in his book titled **Decoding AI**, in which he tries to enable people to go beneath the surface of artificial intelligence and learn the underlying concepts of modern AI.

Preface

AI is all around us. We use it to generate images, to give answers, recognize objects, provide recommendations, and much more.

But have you ever asked yourself how it really works? This book starts from this very point.

A Mind Made of Data is a book about what lies beyond the screen. We will begin our journey from intelligence and data, and proceed with Machine Learning, math, neural networks, computer vision, NLP, generative AI, and other contemporary AI approaches.

No prior experience with these topics is required to start reading this book. All you need is just some curiosity and questions.

One question should always be on your mind when reading this book:

Why does it work?

Once you start asking this question, everything becomes easier to understand.

How to Use This Book

This book is meant to be read, explored, and questioned.

Each chapter contains different sections to help you understand the ideas and apply them.

Concept

The main idea being explained.

Example

A simple example that connects the concept to something concrete.

Figure

A visual explanation of how a system or process works.

Think

A question that gives you a chance to stop and reason.

Try It

A small activity to help you apply what you learned.

Key Takeaways

The most important ideas from the section.

Challenge

A harder question that takes the idea one step further.

You do not have to understand everything immediately. Some ideas will become clearer as you move through the later chapters.

Read. Think. Question. Try.

That is the way I hope you experience this book.

Contents

Chapter 01 Introduction to the idea of Intelligence

- 1.1 Understanding Intelligence
- 1.2 Human Intelligence
- 1.3 Artificial Intelligence
- 1.4 AI and Automation

Chapter 02 The World of Data

- 2.1 What Is Data?
- 2.2 Representing the World
- 2.3 Data Quality and Bias

Chapter 03 How Machines Learn

- 3.1 The Idea of Machine Learning
- 3.2 Learning from Examples
- 3.3 Types of Machine Learning
- 3.4 Learning Through Patterns

Chapter 04 The Mathematics Behind AI

- 4.1 Functions and Representations

4.2 Vectors and Matrices

4.3 Probability and Statistics

4.4 Gradients and Optimization

Chapter 05 *Building a Machine Learning Model*

5.1 *Defining the Problem*

5.2 *Preparing the Data*

5.3 *Training the Model*

5.4 *Evaluation the Model*

Chapter 06 The Artificial Brain

6.1 The Artificial Neuron

6.2 Building Neural Networks

6.3 Learning in Neural Networks

6.4 Deep Learning

Chapter 07 When Machines Learn to See and Understand Language

7.1 Learning from Images

7.2 Learning from Sequences

7.3 Attention and Transformers

7.4 Understanding Different Forms of Data

Chapter 08 Modern Artificial Intelligence

8.1 Generative AI

8.2 Foundation Models

8.3 Multimodal AI and AI Systems

8.4 Intelligence We Build

A Word to the Reader

When considered as a combination of all those new concepts, algorithms, and technologies, artificial intelligence may appear quite puzzling. This book adopts a different perspective on this issue.

We will not consider AI as a mysterious black box but will gradually deconstruct it to see what concepts and notions lie behind it. We will start from data, study ways to manipulate it, will then move on to algorithms, machine learning, neural nets, and current AI.

You don't have to understand everything at once; the aim is to develop a knowledge base in small steps, connecting the dots and learning to see AI as a fascinating field rather than just as a technology.

Let's start then.

Chapter 01

The Idea of Intelligence

Let us visualize presenting a child with three pictures.

One depicts a cat. Another one is of a dog. And the third one is of a bird.

No explanations regarding a cat are needed. There is no need to provide a set of definitions such as "cat has four legs" or "a dog typically has a bigger body." The kid will be able to observe the pictures, compare them with his/her experiences, and recognize the differences.

Let us now imagine a computer trying to perform the same task.

A computer, unlike humans, is not able to perceive a cat naturally. Everything is transmitted through numbers. An image consists of pixels. Sounds form signals. Sentences become sequences of symbols or numbers.

However, contemporary AI systems are capable of object recognition, speech recognition, video recommendation, medical imaging analysis, and text generation.

Then, what exactly do we mean by calling something "intelligent"?

This question precedes Artificial Intelligence.

What Is Intelligence?

Intelligence is one of these words which seem very clear but become complicated when trying to give a definition.

For example, a student who succeeds in solving a complicated math problem can be considered intelligent. Similarly, a person who learns a new language rapidly, a researcher who uncovers some patterns, and a man who finds a solution to a new problem are also regarded as intelligent.

It is obvious then that intelligence is not a skill.

There is no universally accepted definition for intelligence and all fields have studied this term from their perspective. Intelligence has been investigated from psychological, neurological, philosophical, and computer science points of view. Nonetheless, most of the discussions about intelligence include skills like learning, reasoning, problem-solving, perception, adaptation, and generalization.

Learning is about acquiring information or improving performance through experience.

Reasoning refers to making conclusions based on information.

Problem-solving involves the process of going from the present state to the desired one.

Perception helps an intelligent system in interpreting information from the environment.

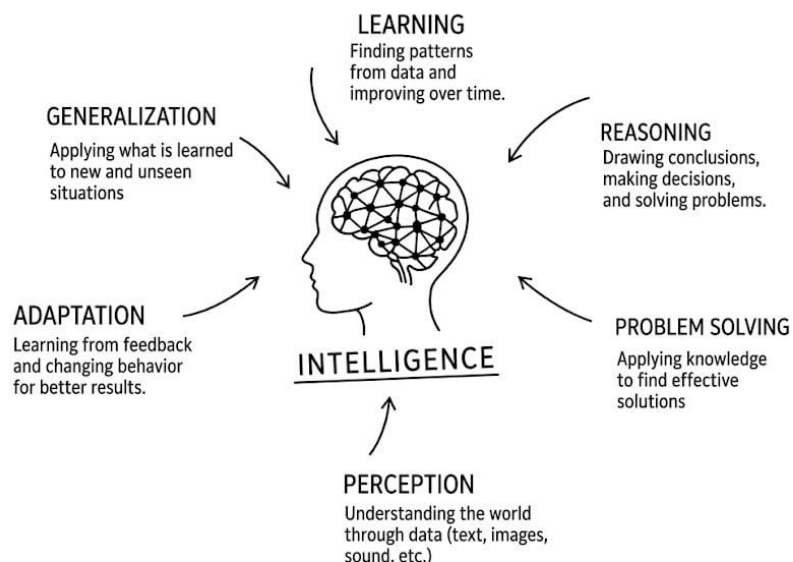
Adaptation is about changing behavior in changed conditions.

Generalization is the ability of using past experiences in new situations.

These skills are interrelated.

It cannot be said that the individual learns how to ride a bicycle through the rote learning of information. What they do is observe what takes place, make corrections, and ultimately become proficient in something they've never done before.

Such a capacity to gain knowledge and use that knowledge is one of the concepts that acquires special significance as we study machine learning.



Information Alone Is Not Enough for Intelligence

Information is certainly valuable, but it doesn't automatically translate into intelligence.

A computer database may have millions of pieces of data stored within. This could be information on nations, mathematical formulas, chemicals, historical events, or even new scientific findings.

However, storing such information is quite different from applying it.

For example, a student memorizes all the formulas in a math book but fails to understand how they should be applied to solve certain problems. Thus, he knows many formulas but still doesn't know which one to use for a particular situation.

Intelligence means being able to apply information.

The reason this becomes important for artificial intelligence research is because computers can store immense amounts of information. Having much storage space is not the same as having intelligence.

The crucial point is the ability to utilize that information.

A Simple AI Example

Take an example of spam classification in emails.

A simple system may be based on an explicitly written rule such as: "If an email has some specific word, call it spam."

This is effective when the problem is really simple. However, emails are much more complex. Emails may contain the same message written in different ways, emails may contain strange spellings, etc. Besides, sometimes people send emails that seem legitimate, but they actually are unwanted.

In a more sophisticated system, the patterns are learned using many examples.

Instead of programming all possible rules into the system, we can just show many examples of emails that were classified as spam/not spam. Based on these examples, the system will learn some statistical patterns to perform classification.

This difference between explicitly specified rules and learned patterns is going to be one of the key concepts of this book.

Human Intelligence

The human being is the best-known instance of an intelligent system.

Think of yourself walking into a room that you have not seen before.

In seconds, you would be able to discern things.

You will be able to recognize a chair, a door, a window, a desk, and people.

You will be able to judge the relative positions of the things.

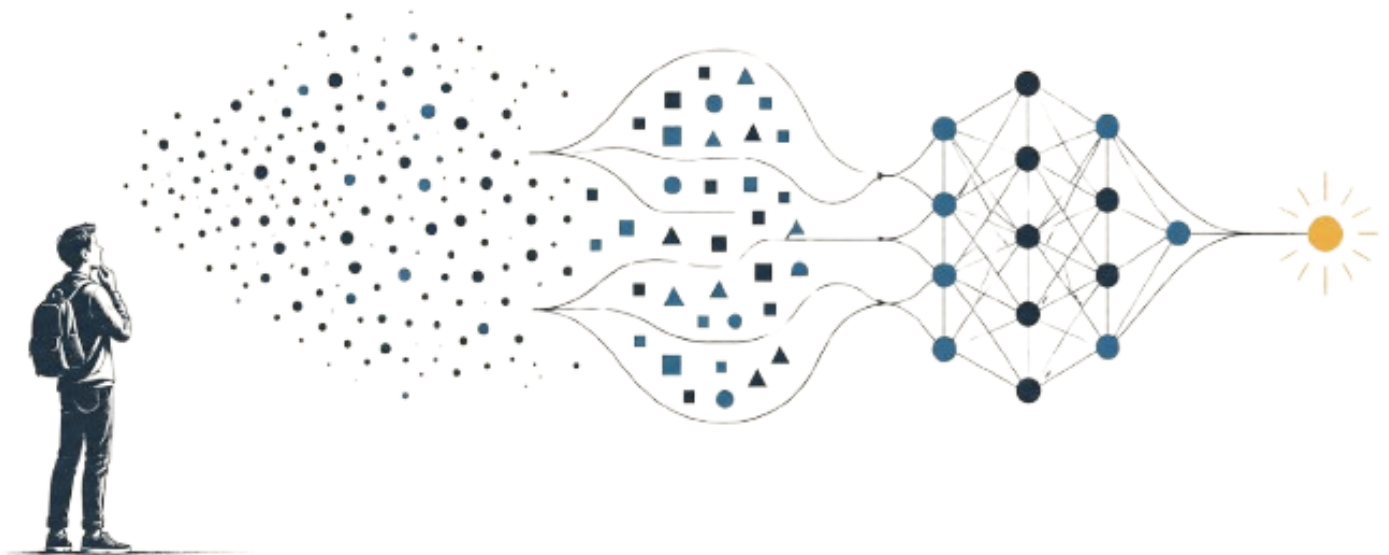
You will be able to recall rooms that are similar.

You will be able to predict that opening up the door might take you somewhere else.

You will be able to decide on where to sit.

None of this would require you to figure out each of these things consciously.

Your brain integrates all these things from perception, memory, prior experience, reasoning, and decisions.



Learning from Experience

One of the key features of human intelligence is the ability to learn from experience. Imagine an individual with no previous experience in a certain game. At first, many mistakes will be made. With enough practice, however, patterns will emerge. One will learn what works and what does not, thus changing the strategy. With enough practice, one will perform better compared to his/her initial state. The important thing is not memorizing all cases, but learning patterns that help to deal with new cases. This idea comes up often in the field of Artificial Intelligence; modern AI systems often learn from examples and not from exhaustive instructions for each situation.

Generalization

The value of learning becomes apparent when it helps to deal with previously unseen situations. Suppose that a student solves ten quadratic equations. She/he can either memorize answers for those ten specific equations, which gives little value, or learn the pattern of how such problems should be solved, and solve any new quadratic equation that has not been seen before. This ability is called generalization. Generalization is the main goal of machine learning: a model should find patterns that go beyond the examples used for training.

Is there one intelligence or several different intelligences?

Imagine two systems. First, it shows extraordinary skills at playing chess by evaluating millions of possible positions and making a good choice. Second, it possesses a wider range of skills like object recognition, understanding of language, navigation in unknown environments, learning of new tasks and communicating with humans. The first system may demonstrate good results only in its narrow domain but not be able to work in another one. That is why we get an important difference.

Specialized Intelligence

The system that was designed to deal with one certain task can show great results in it. Here are some examples of specialized intelligences:

- Chess algorithm
- Spam filter
- Image classifier
- Recommendation engine
- Speech recognizer

Those systems were built based on certain problem space and cannot perform other tasks without any changes.

General Intelligence

General intelligence is understood as a wider set of skills that allow learning, reasoning and solving different types of problems. Human being is a perfect example of such ability – people can learn math, speak languages, recognize objects, navigate through environment, create arts and deal with unusual situations. The difference between specialized and general intelligence is very important for AI discussions.

Specialized Intelligence

Excels at a specific task

An AI system can be designed to perform extremely well at a narrow domain, such as playing chess.

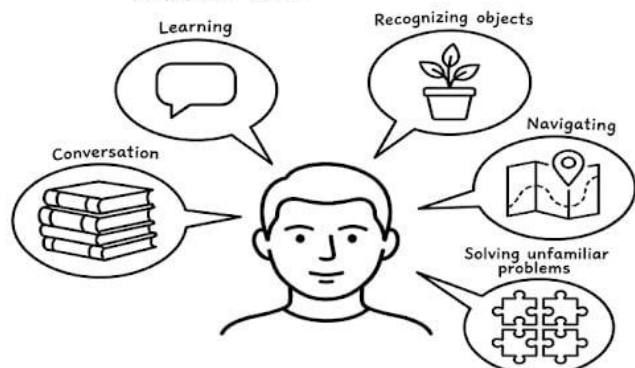


Specialized AI = High performance in a limited area.

Broad Intelligence

Handles many different tasks

Humans can understand, learn, adapt, and solve a wide range of problems in the real world.



Human intelligence = Flexibility, creativity, and real-world understanding.

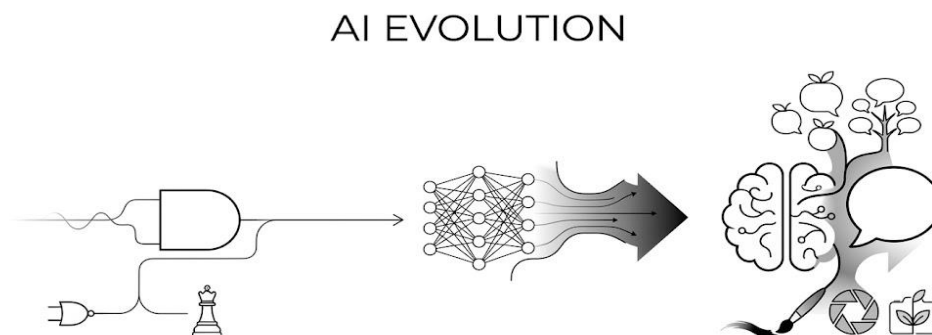
Human Intelligence to Artificial Intelligence

The notion of intelligence being a set of capabilities prompts another interesting question: can some of these capabilities be achieved through machines? This very question has played a major role in the rise of the field of Artificial Intelligence (AI), which, in common computer science parlance, refers to a domain whose goal is the development of computing systems that can perform operations linked with capabilities like perception, reasoning, learning, natural language processing, planning, and decision making.

It should be noted that the definition of AI makes it clear that there is no necessity for any attempt to build a machine whose way of thinking would be just like that of the human brain. A computer does not have to mimic all the biological operations of the brain in order to do something intelligent. Similarly, an airplane need not simulate the flapping of wings in birds to fly. The same can be said about the AI technology: its way of accomplishing tasks related to intelligence need not be like that of human beings.

Artificial Intelligence Over Time

Artificial Intelligence is an area of study which should not be perceived as one technology that appeared suddenly in modern times. This field of study has developed over the years by means of several different key paradigms. At the very early stage of development of artificial intelligence, scientists used methods of symbolic reasoning, search and logic, together with technologies based on explicit rule-based representations of knowledge. Later, attention was paid to statistical methods of analyzing data, making machine learning a key paradigm. Computing technologies, the availability of large amount of data and development of neural networks have significantly contributed to the development of deep learning.



Machines that Obey Rules

Before the advent of the ability of machines to detect patterns within huge datasets, there have been many smart systems built on rule-based principles. For example, consider building a simple system for detecting whether the e-mail is suspicious or not. You could write some rules like:

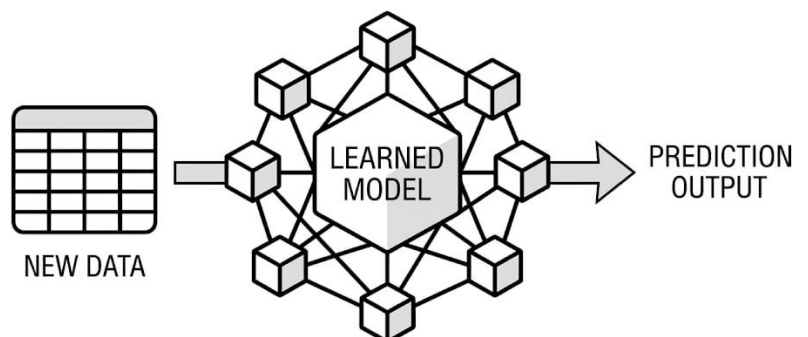
- Increase the suspicion if the sender is unknown;
- Increase the suspicion if the message includes some patterns;
- Mark the message as spam if there are several suspicious characteristics.

Such approach works well and is very interpretable as you can look through rules yourself. But there is one big problem here. The reality is full of possibilities that cannot be covered by just the list of rules. The human language constantly evolves, with new spellings and expressions being introduced. Also, there might be some new kind of spam. A machine that is based on the set of hard-coded rules will have problems with situations that developers couldn't foresee. And that is the main reason for the transition to learning from data.

Rule-Based Systems vs. Machine Learning

There are two fundamentally different paradigms for solving problems. According to the traditional rule-based paradigm, the person defines the rules. With the machine learning paradigm, the person provides examples, and the algorithm learns rules from them. It could be put like this:

AI EVOLUTION: CONSOLIDATED OUTCOME



Traditional Approach

Data + Rules → Output

Machine Learning

Data + Expected Outputs → Learnt Model

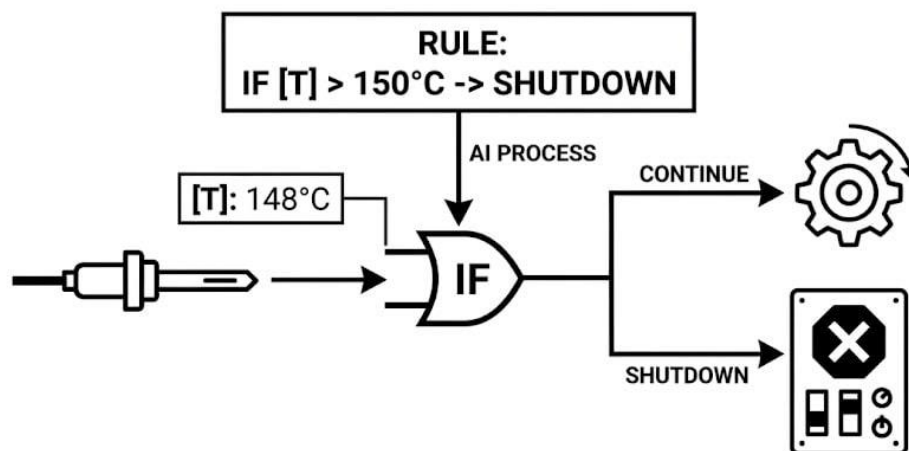
And then: New Data + Learnt Model
→ Prediction

It is not meant that machine learning has completely replaced the rules. There are many applications where rule-based systems still work well. The main difference is in the source of the behavior. In the case of rule-based systems, humans specify their logic explicitly, while in the machine learning case, the model learns rules from examples. It will become clearer in the chapter 03.

AI and Automation

There are many misconceptions about what artificial intelligence is. In particular, artificial intelligence is often considered a synonym for automation. However, there are differences between these two concepts. To begin with, automation is when technology is used to carry out a process with minimal human involvement. As an illustration, a factory machine detects the temperature of a motor and acts based on the following principle: if the temperature goes above 80 degrees Celsius, then the machine stops working. This does not involve any understanding of how a motor works; the system just follows the defined rule.

Now imagine a different scenario: a camera watches products passing by on the conveyor belt, analyses the images, and predicts whether the product has a flaw or not. The key difference is that there is no one-size-fits-all rule to define all kinds of flaws; instead, the system can learn patterns using multiple samples of flawed and flawless products. It is in this way that artificial intelligence can be introduced into an automated process.



The key idea is simple:

Automation can follow instructions. AI systems can also use learned patterns to make predictions or decisions.

The boundary is not always perfectly sharp. An automated system can contain AI, and an AI system can be used inside an automated process.

A Day with Artificial Intelligence All Around

It is not required to visit any futuristic laboratories in order to come across Artificial Intelligence because AI systems are common to our everyday lives.

For example, face recognition to unlock a phone uses vision recognition; navigational services suggest the ways using data analysis algorithms that process the information about roads, places, and traffic. Video recommendation sites use models to recognize patterns in the users' behavior and content. Spam filtering uses classification models to recognize any abnormal patterns. Language models produce text according to the large computation models of language. Medical software used to analyze radiological or other medical pictures is able to help the doctors to recognize some important patterns for them to investigate.

While the technologies vary from one sphere to another, the logic behind the whole thing stays the same: The information goes into a computational system and comes out as some kind of output.

A Scientific Analysis: How Does the AI Recognize a Cat?

In terms of the above case, an artificial intelligence model is fed a picture of a cat. What occurs inside the machine?

At a basic level, the digital image is comprised of numbers. Each pixel represents information on features like color and brightness. The algorithm doesn't learn the definition of "cat"; it learns from data.

A machine learning algorithm is taught using many samples, where each image is provided together with its label. During training, the parameters of the model are updated for it to be better at recognizing patterns related to different categories. After the training stage, a new image can be inputted to the model, and the algorithm will use this data to make a prediction.

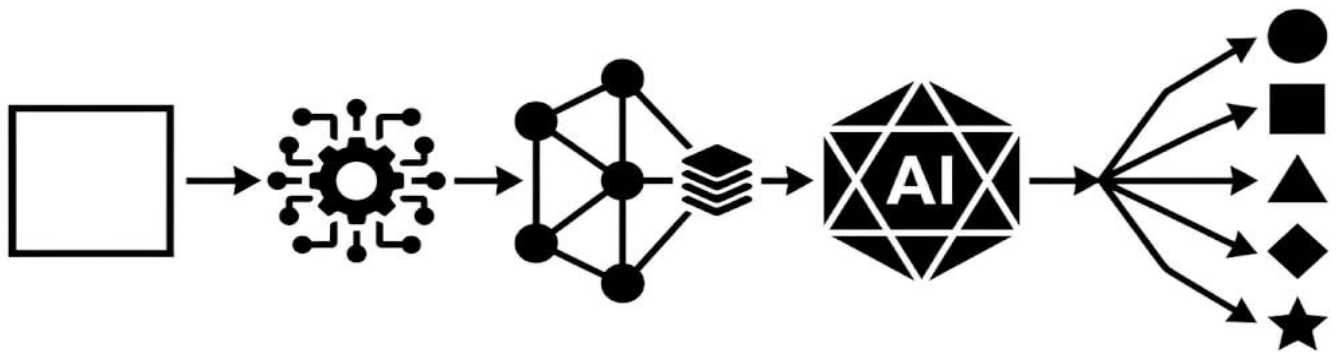
For illustration:

Input: Image

Model: Visual Patterns Learned by the Algorithm

Output: Cat + Confidence Score

Though the mechanism can be very complex, the core idea remains simple: the algorithm uses patterns it had previously learned to make predictions about new data.



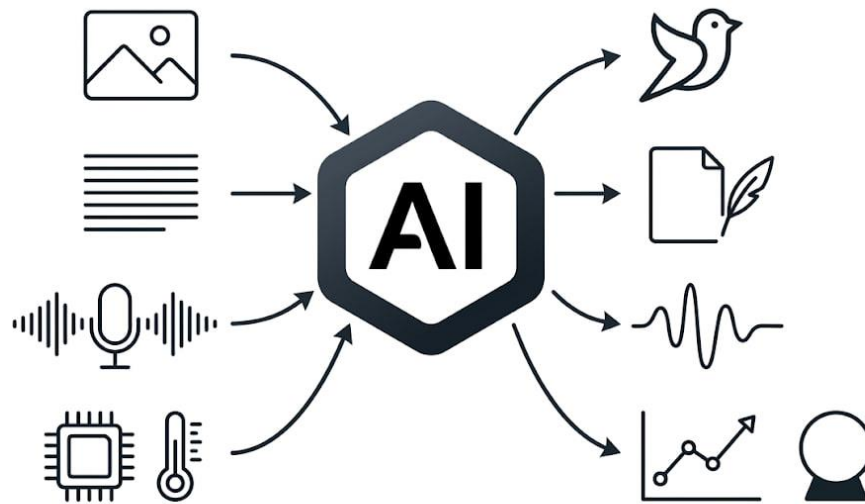
In this passage, a premise is introduced that will be carried forward as one of the themes in the entire book: data forms the base on which a considerable amount of today's AI systems learns.

The Input-Output Structure of AI Systems

There are many AI systems that can be generalized into an underlying structure based on the order: Input → Model → Output

Here are some examples that follow this structure:

- Image as input and classification as output.
- Sentence as input and output response as output.
- Sensor values as input and prediction as output.



Later, we will ask:

How is the input represented?

How does the model learn?

What mathematical operations happen inside it?

How do we know whether its prediction is correct?

And what happens when the model makes mistakes?

Those questions will lead us from AI into data, machine learning, mathematics, neural networks, and modern AI systems.

Artificial Intelligence, Machine Learning, and Deep Learning

Three concepts can be often found in current discussions:

Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning (DL).

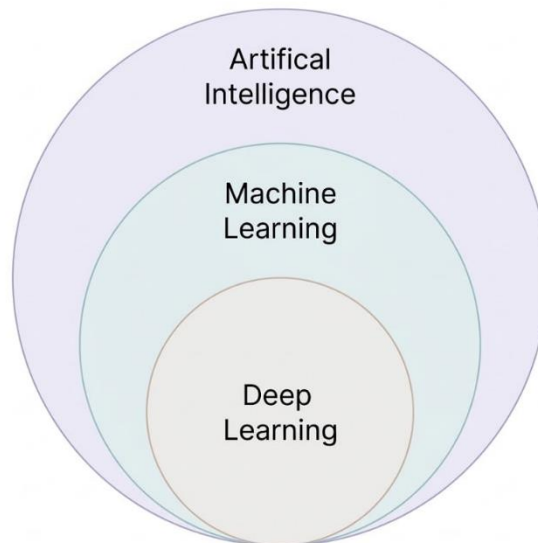
These are linked but different notions.

AI is the most general concept.

In artificial intelligence, one of the key methodologies is machine learning where machines learn from data.

Deep learning refers to a particular kind of machine learning algorithms using artificial neural networks with many layers.

One can easily imagine these notions as a nested pyramid.



This distinction will matter throughout the book.

When we talk about AI, we are talking about the larger field.

When we talk about machine learning, we are focusing on systems that learn patterns from data.

When we talk about deep learning, we are focusing on neural-network-based approaches that can learn complex representations from large amounts of data.

A Medical Example

Let's take medical image recognition.

A physician might analyze an X-ray or other medical image in search of any patterns that might indicate some disease.

The artificial intelligence system can be trained using a large number of labeled medical images.

The system learns certain patterns related to the examples during training.

Then, upon receiving a new image, the system will make a prediction.

The prediction will not replace the physician.

Depending on how the system was developed and what it will be used for, artificial intelligence can be regarded as an instrument that allows detecting some patterns, prioritizing cases, or providing extra information.

However, there is one more thing that we learn from the example above.

The performance of the artificial intelligence system highly depends on the data used in the development process.

In case the training dataset is incomplete, biased, or improperly labeled, the algorithm will make faulty predictions.

Therefore, we cannot end the story of artificial intelligence with the model itself.

It is the data that really matters.

And it is where our story continues.

Bigger Picture

We began with a single straightforward question:

What is intelligence?

We found out that intelligence isn't a single trait.

It may consist of learning, perception, reasoning, problem-solving, adaptation, and generalization.

These traits are exhibited by people in various contexts.

Artificial Intelligence strives to create computational systems capable of accomplishing some of these tasks.

Some AI systems use rules.

Some others learn rules from data.

Machine learning enabled the creation of such systems capable of learning from data instead of manually defining all the rules for them.

Deep learning took it a step further through the creation of multilayer neural networks capable of learning complicated data representations.

Nowadays AI systems can deal with various kinds of data, including images, text, audio, sensor data, and other.

But there is still one question left.

If machines can learn from examples, where do these examples come from?

They come from data.

And before we start talking about machine learning, we have to talk about data first.

Big Idea

Intelligence isn't just an ability to memorize data.

It is an ability to learn, reason, adapt, and solve problems using data.

Artificial Intelligence attempts to recreate some of these features through computational systems.

Machine learning gives those systems the ability to learn patterns from examples.

And those examples begin with data.

Next Chapter:

Chapter 02

The World of Data

Before a machine can learn from examples, we need to understand what those examples actually are.

That begins with data.

Chapter 02

The World of Data

Modern artificial intelligence is predicated on one simple principle: Machines are able to learn from information that has been represented in a way that allows them to use it.

The way that information is represented will usually determine what a learning system is able to discover, predict, and fail to discover or predict.

This means that data is more central to artificial intelligence than might first be thought. In talking about algorithms, neural nets, or large language models, there is an important step that needs to be taken first.

Data can be used to represent virtually anything. Numbers represent measurements. Sequences of numbers represent signals. Matrices represent images. Tokens represent sentences. Records represent events, whereas datasets of thousands or millions of records represent trends across many instances.

The key here is that data isn't the thing itself; it is a representation of something else for a particular reason.

It is around this distinction that this chapter is built.

What Is Data?

Data can generally be described as recorded information which can be processed or analyzed.

The term data is deliberately defined quite broadly. Although computers operate with information using numeric and symbolic codes, data is not necessarily numbers.

A thermometer can record the following series of values: 24.8, 25.1, 25.0. A camera creates information in the form of a picture of pixels. A microphone records the variations of air pressure over time. A document contains symbols and words that can be processed computationally.

They are completely different kinds of information, but still they can all be turned into data.

That makes data useful is not just its existence, but rather its connection to something which we want to investigate or predict. Measurement is meaningful since we know what we have been measuring. The value in a column of a data file has some meaning since we know what this column stands for.

Consider the number:

72

By itself, it tells us very little. It could represent a student's score, a person's age, a temperature in a particular unit, a distance, or something else entirely. The numerical value is the same, but its meaning changes with context.

For this reason, working with data always involves two questions: **What was recorded?** and **What does it represent?**

This becomes particularly important in artificial intelligence because a learning algorithm does not have direct access to the physical world. It operates on the representation supplied to it.

From Reality to Representation

Let us take the example of a weather monitoring system. There are countless physical parameters present in the atmosphere which may be impossible to measure or to measure continuously. As such, the weather monitoring system takes into account some specific physical parameters that serve its purposes, such as temperature, humidity, pressure, and wind speed.

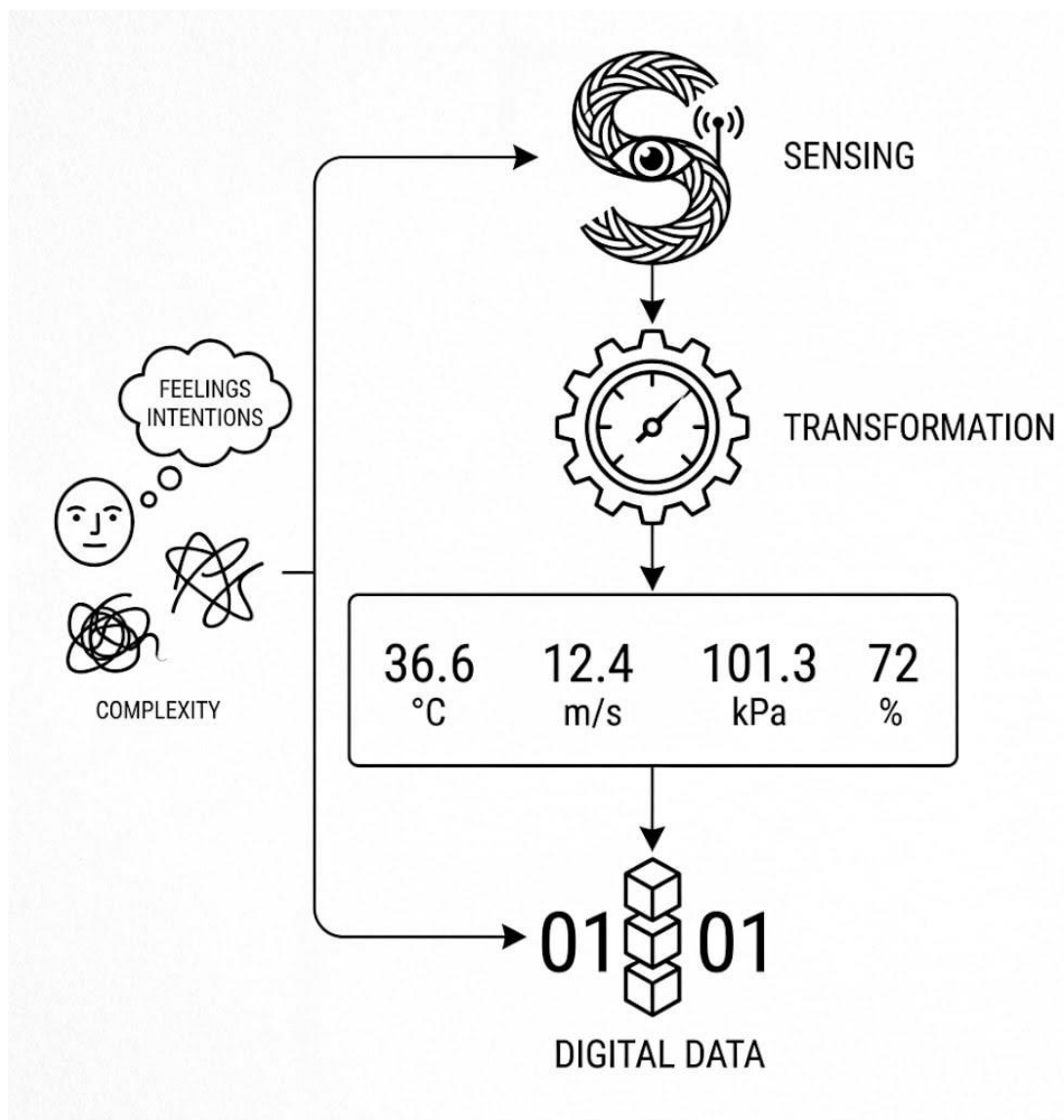
These parameters are measured using sensors which transform the physical phenomenon into signals which are represented by numbers.

For instance, some data points in the dataset may be temperature = 28.4°C, humidity = 64%, and wind speed = 12 km/h.

The numbers are valuable since they represent measurable parameters of the environment. But they are just incomplete representations. The dataset does not include all the parameters of the atmosphere. It just includes certain parameters observed through a specific system of measuring.

And this is what we get from it, one of the most important principles of the whole book:

Data is the representation of reality but not reality itself.



Observations and Datasets

A single observation may be helpful, but machine learning requires multiple observations.

Assume that a researcher intends to analyze house prices. A single house constitutes a single observation. Thousands of houses constitute a dataset from which statistical relationships may be estimated.

A dataset is a set of related observations designed for a particular computational or analytical purpose.

The form of the dataset depends on the problem being solved.

In a house price prediction problem, the observation may include information about the size of the house, the number of rooms, the location, the age, and the price. In an email classification problem, the observation may include information extracted from the email along with a classification of whether it is spam or not. In a picture recognition problem, each observation will mainly be made up of an image and a label.

The use of the term "observation" helps us keep in mind that every row or example in a dataset represents some observation, recording, or construction.

A dataset is thus not just a set of numbers. It is an organization of observations.

This point becomes essential when we start asking how a model learns from those observations.

Features and Labels

Many machine learning problems can be described in terms of features and labels.

Feature is a description, characteristic, or measurement that is used to describe an observation. For instance, for a house-price problem, the area of the house, the number of rooms, and the age of the building could all be features.

Label is the target for the learning problem when the learning problem has a known answer that the model is expected to predict.

For example, say we have:

Area= 150 m²

Rooms= 3

Age= 7 years

Location= Cairo

and the recorded selling price is 2.8 million EGP. The first group describes the house and the selling price is the target. Using mathematical notation, we can describe the features of an observation by a vector:

$$\mathbf{x}=(\mathbf{x}_1,\mathbf{x}_2,...,\mathbf{x}_n)$$

and the target with: $\mathbf{y}$

A supervised learning algorithm tries to learn a relation between $\mathbf{x}$ and $\mathbf{y}$. In a simplified form, we can write:

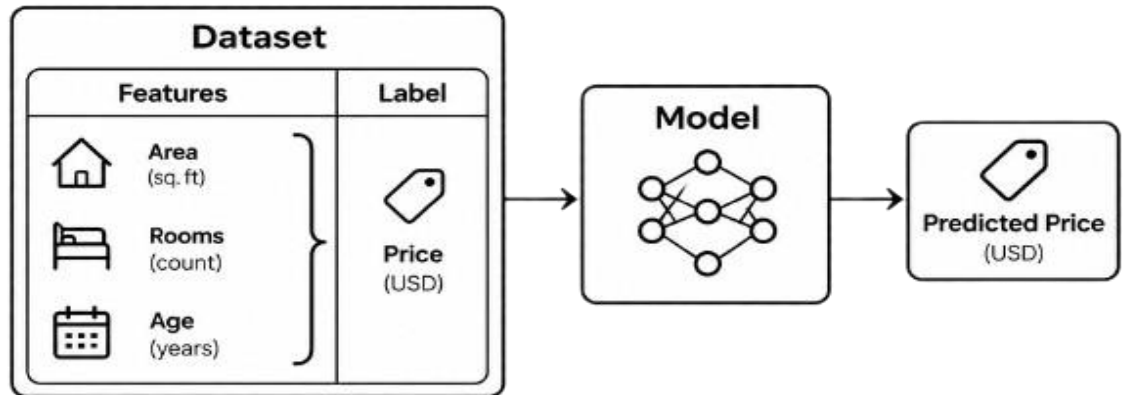
$$\mathbf{y}^{\wedge}=\mathbf{f}(\mathbf{x})$$

which means that the model predicts $\mathbf{y}^{\wedge}$, that is, the model's output given the input $\mathbf{x}$. Here $\mathbf{f}$ is the function that the model learned from the training set that we will see in later chapters.

The purpose of this notation is not to make the problem unnecessarily mathematical but to introduce some terms that will be essential in later chapters when we learn about functions, models, vectors, and optimization.

The same structure appears in many problems.

For classification problems, the label can be a category, for instance, for spam detection, the label can be "spam" or "not spam." For regression problems, the label can be a quantity or a number, for instance, a house price or the temperature. In image classification, the features are derived from the image representation and the label can be "cat," "dog," or any other class. The model receives information about an observation and tries to find a function or relation between that information and a target.



The distinction between features and labels also reveals an important property of supervised learning: the system is given examples for which the desired output is known. But that is not true for every machine learning problem.

When the Answer Is Not Given

Let's say that we have thousands of customer records, keeping track of what they purchased and how often. We have some information about how much they spent, but we haven't classified each customer into a specific category.

Does that mean there isn't anything else we can do with this data?

Well, there is still information in there! People tend to clump together into different categories based on some similarity, and we could try to find those similarities - we just won't be getting a label for every single observation telling us what to do with it

This concept is called **unsupervised learning**.

In supervised learning, our training data has a target variable that we're trying to predict.

In unsupervised learning, the algorithm will try to find structure in the input data.

Clustering is one example of an unsupervised algorithm.

By applying a clustering algorithm, we can try to group similar observations together, even if we don't know what those groups are ahead of time. This makes a big difference in how we need to think about the problem.

The purpose of a supervised machine learning model is to learn the relationship between input variables and a target variable. On the other hand, an unsupervised model has to learn structure from the input data itself. Later on, when we learn about machine learning more formally, we'll see how these concepts tie in.

Different Forms of Data

Data can come in all different formats

There's structured data, which is data that has a specific organization to it. A database is a good example of structured data - we have rows and columns, with each row representing an observation and each column representing a feature of that observation. There's also unstructured data, which doesn't have a clear way of being represented. Documents, images, audio, and video are all examples of unstructured data.

Text documents can take many different forms, and video and audio data have their own unique presentation. The term unstructured is something of a misnomer, since all data has some structure to it. Images have spatial structure in how the pixels are arranged, audio data has a temporal structure, and text data has a grammatical structure. The main difference between structured and unstructured data is that a database or spreadsheet is structured in a way that it can be fed directly into a machine learning model, whereas other forms of information require us to find some way of transforming the data into something our machine learning model can understand

It's important to think about what form the data takes, since it will influence how we can represent it. A numerical dataset can be represented as a vector or matrix, but an image could be represented as a 3D array of pixel values. Audio data can be represented as a time series, and text data can be represented as tokens and a vector of numbers. Graphs can be represented as connections between nodes and edges. We'll see later on that the type of information we have will affect the type of machine learning model we decide to use.

Representing the World

A computer doesn't think in terms of abstract concepts. It only works with representations of information, and this is a core challenge in artificial intelligence

Let's take an image as an example. A human might be able to look at an image and identify objects and shapes, but a computer vision application has to start somewhere else - with a numerical representation of the image. An image can be represented as a matrix or a 3D array, depending on how the computer stores the data. Each pixel in an image is represented with numbers that represent each color channel. For example, a grayscale image would have one value per pixel, with each value representing the "lightness" of that pixel. A color image has 3 values per pixel, representing the red, green, and blue channels. A 3-channel image that is 100x100 pixels can be represented as a 3D array:

$100 \times 100 \times 3$

There are 100 x 100 pixels in each dimension, and 3 values for each pixel representing the different color channels. Does this mean that the computer "understands" what's in the image? No, not really. All the computer can do is perform computations on the data, which requires that we represent the data in some mathematical form. That's not to say there isn't any understanding happening - but a machine learning model isn't going to understand a document any better than you do when you first read it. It just has a representation of the document that it can perform calculations on.

This is a similar concept applied in language. A computer doesn't process language the same way that humans do. A language model needs some way of representing the text so that it can perform operations on the text. A natural language processing (NLP) model often breaks the text down into tokens, and then transforms those tokens into a vector of numbers that can be fed into the model. The implementation details can vary from task to task, but the overall problem is the same: how can a computer understand representations of real-world objects? This idea will show up again and again in the coming chapters.

Representation Determines What Can Be Learned

The choice of representation determines what patterns can be learned.

Imagine a system for recognizing objects in images.

If the input consists of a limited and incomplete description of each image, then useful information will not be available to the model.

Or suppose a model is trained on a narrow range of conditions.

Patterns that emerge during training may generalize better within the range, but worse outside it.

Representation has two aspects.

The first is computable information, which means that the data has been encoded in a way that an algorithm can process.

The second is that the representation captures information that is relevant to the task.

This is why feature engineering was historically a crucial part of machine learning: engineers had to determine which measurable properties would be useful to an algorithm for making predictions.

Modern deep learning systems are able to learn useful representations automatically in many situations, but this does not change the fact that representation is still critical.

Just as learning is affected by what can be observed, it is also affected by what can be represented.

We will reconsider this topic in the context of neural networks and representation learning.

From Raw Data to a Learning Dataset

Raw data is rarely in a form that can be used directly.

Measurements may be missing.

Values may have been recorded incorrectly.

Different units may be used in different sources.

Some observations are duplicates of others.

Labels may be incorrect.

Some variables may be completely irrelevant to the task.

Consider the following list of body-temperature measurements:

36.7°C, 36.8°C, 36.6°C, 367°C, 36.9°C

In this example, 367 is suspiciously far from the other values.

It could represent a decimal-point error, a faulty measurement, or some other issue.

A learning algorithm has no way of knowing which is the case on its own.

This is where data preparation comes in.

The process could involve verifying the data, filling in missing values, removing or fixing erroneous records, transforming variables, generating features, and ensuring that the representation is suitable for the task.

These are not minor housekeeping issues.

They directly impact what the model can learn.

Data Quality

The quality of a dataset cannot be determined by the quantity of examples.

A good dataset is one that is appropriate for the task and contains enough information to support the inferences or predictions we want to make.

There are several dimensions to concern ourselves with.

Accuracy

Accuracy indicates how well a recorded value reflects what it claims to measure.

If a sensor gives the value **367°C** when the actual temperature is **36.7°C**, then the recorded value is not accurate for the intended purpose.

Completeness

Completeness indicates the extent to which the data includes all the information that we want to capture.

A dataset may contain thousands of records about houses but be completely useless for a particular task if it lacks a critical variable.

Consistency

Consistency indicates whether values are recorded in compatible ways.

If a dataset mixes units of measurement without indication, then it will contain consistent-looking values that have different meanings.

Relevance

Relevant data contains information that is useful for the task.

A dataset can have a huge number of variables but not contain any of the ones that we actually need.

Timeliness

Timeliness indicates whether the data represents conditions as they actually exist.

If we train a model using historical information and then use it in a different environment, it may not behave as we expect.

If the conditions changed, we may be using data that is no longer representative.

These qualities are not entirely independent, but it is important to remember that a dataset can be accurate and incomplete, or complete and irrelevant, or large and unrepresentative.

The purpose of the dataset must guide our thinking.

Missing Data and Measurement Noise

Missing values are common in practice.

A sensor may not have captured a measurement.

A participant may have left a question blank.

A database might have incomplete entries.

An experiment might not have collected the necessary information for some observations.

What we do in these circumstances depends on the cause of the missingness and the importance of the variable.

Options include removing observations, estimating the value, using algorithms that tolerate missing information, and collecting the missing information.

The choice is not always obvious, but the reason for the missingness itself may contain valuable information.

Suppose a survey is given to students about a particularly challenging assignment, and students who struggled are less likely to fill it out.

The missing responses are not random.

Replaying them with an average would alter the dataset in a way that does not reflect reality.

Another common issue is noise.

Recordings often contain small inconsistencies due to sensors, environment, sampling, and other factors.

If a temperature sensor gives values of **25.0°C**, **25.1°C**, **24.9°C**, and **25.2°C** while the environment remains approximately the same, then the variations are not necessarily significant.

We need to learn to distinguish between variation that indicates meaningful change and variation that does not.

This will become more important later when we consider generalization and overfitting.

A model that memorizes random fluctuations will perform well on the data it was trained on, and poorly on new examples.

Sampling and Representation

Most datasets are not descriptions of every possible observation.

They are usually constructed from a sample of a population of interest.

Let's say a researcher wants to determine a characteristic of students in an entire country.

It may not be practical to measure every single student, so a sample is taken instead.

The value of this approach depends on how the sample was chosen.

If the sample comes from a narrow category of students, it may not be a good representation of the population of interest.

This distinction creates two related but different concepts:

The dataset may be internally consistent.

The dataset may be a good representation of the population.

These are not the same thing.

A dataset can have accurate measurements for every observation it contains, but that does not ensure that it will be useful for understanding a larger population.

This issue is often discussed in terms of representativeness and sampling bias.

If the distribution of the training data is significantly different from the distribution of the test data, then the model will not perform well in practice.

This is one reason why the collection of data is not as simple as gathering as much as possible.

Data Bias

The word bias is used in a variety of statistical and machine-learning contexts, and the exact meaning depends on the usage.

The main concept at the data level involves systematic patterns that are introduced by the process of collection, sampling, measurement, labeling, or construction.

For example, imagine a vision system that is supposed to recognize road signs.

If almost all of the training images were captured in daylight conditions, the model has more to learn about night-driving than about road signs.

The issue is not that the dataset is "biased", but that we need to understand exactly what it's that the dataset is biased towards.

Was the sampling process lopsided?

Were some examples deliberately excluded?

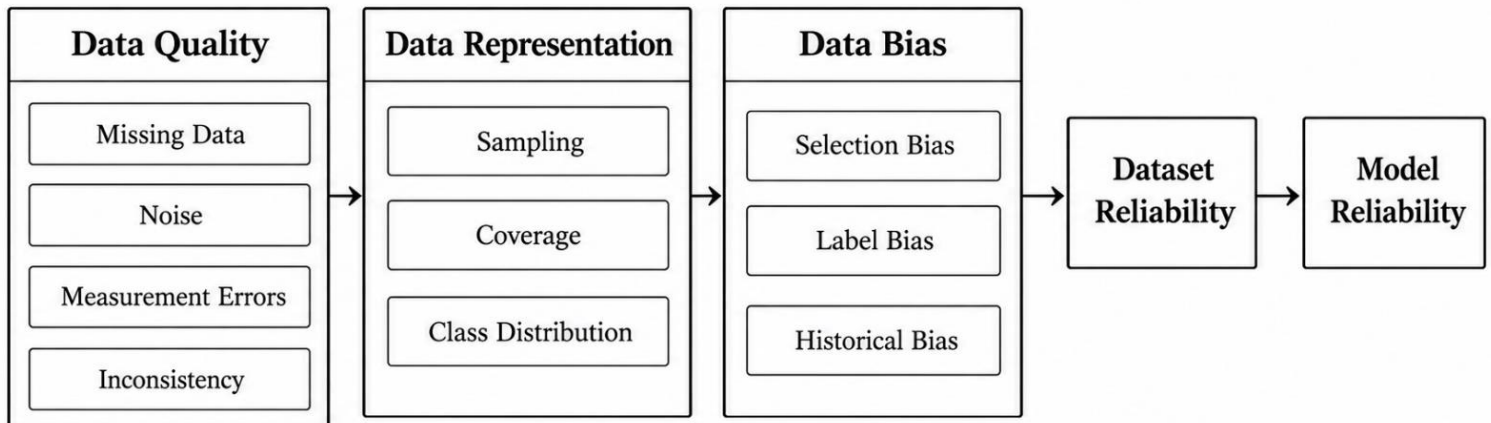
Were labels applied consistently?

Did the measurement process change between sources?

These are all questions we can ask and answer.

Data bias is not an inherently mysterious concept.

Data Quality and Bias



Historical Data

A more challenging situation emerges when a dataset describes decisions from the past.

Historical data describes the conditions and decisions of the system that created it.

If there are systematic patterns in those decisions, then a learning system that is trained on the resulting records will reproduce those patterns.

This does not require intent.

A machine learning model simply learns statistical relationships from data.

Understanding where the data came from is therefore critical.

A dataset is never neutral simply because it is presented in a tabular format.

Labels Are Measurements Too

It is easy to forget that labels are not facts.

In practice, they tend to be the product of some process of observation or even human judgment.

Let's suppose we have an image-classification dataset.

If humans are labeling the images "cat" or "dog", what exactly do they mean?

The results clearly depend on what instructions were given to the labelers and what

decisions they made when ambiguity was present.

They might disagree.

The image might be ambiguous.

The instructions might be unclear.

Some of the errors may be random, and some systematic.

Label quality is therefore another aspect of data quality.

This issue is important because, in the context of a supervised learning algorithm, the targets are treated as information from which the model should learn.

Any systematic error in the targets will therefore become part of what the model learns.

This leads us to an important principle:

The quality of a learning system is limited in its ability by the quality and relevance of the information used to train it.

It is not always possible to improve a model by selecting a more sophisticated algorithm.

Sometimes the more fundamental issue is the data itself.

It may need to be collected in a different way.

The labels may need to be examined.

The representation may need to be adjusted.

The sampling strategy may need to be reviewed.

Data Leakage

One final issue arises when information that should not be available during prediction becomes available during training.

This is sometimes called data leakage.

Let's imagine that we want to build a model that can predict whether a patient will develop a particular condition.

The prediction should be made before the diagnosis, but during training, the model sees data that contains a variable indicating the outcome of the diagnosis.

The result is a model that performs incredibly well, but the performance is deceptive.

When it comes time to make predictions in the real world, the model will not have access to this variable.

If information from the second process enters the training data as if it were available during the first, the evaluation will no longer reflect reality.

This concept is related to how we define the prediction task.

A proper machine learning experiment must preserve the information that is available at the moment of prediction.

More Data Is Not Automatically Better

It is tempting to describe the availability of data quantity as the primary consideration in machine learning.

Quantity certainly plays a role in many situations, but it is only one attribute of a dataset.

Imagine that we have two datasets for the same task.

The first contains one million observations, but the labels are unreliable and include many duplicates.

There are issues with sampling and missing information.

The second has one hundred thousand observations that were carefully gathered, labeled, reviewed, and sampled to reflect the intended population.

The first is bigger.

Does that necessarily make it more valuable?

It depends on what we want the data to do.

This is why data analysis and preparation are such a large part of practical machine learning. A model is only the visible part of the process.

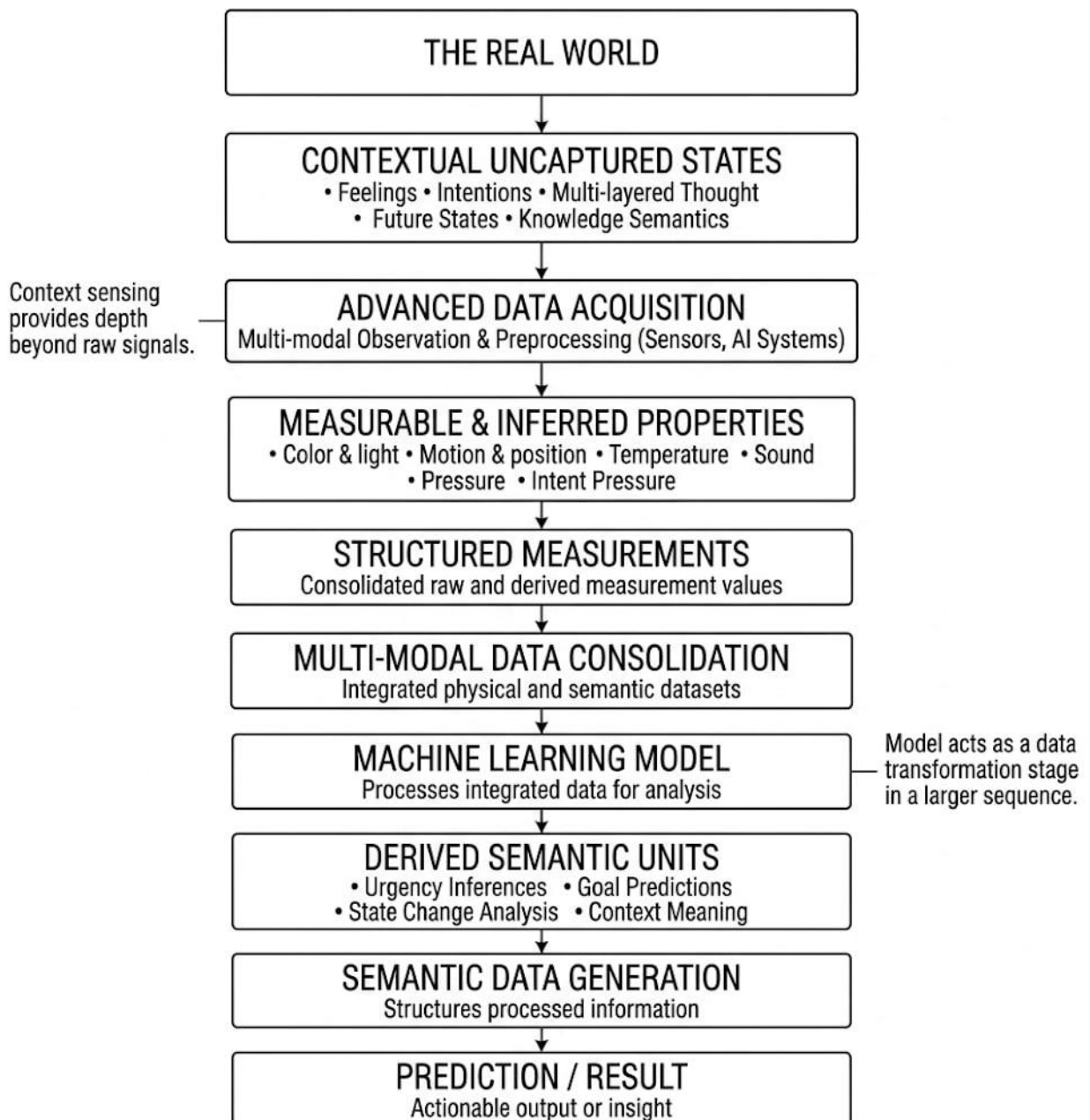
The Data Pipeline

At this point, we can combine these ideas into a single process.

Information begins its journey in the world, and is observed through some measurement or collection process.

The resulting observations form a dataset, which is then inspected, cleaned, transformed, and represented in a way that makes it suitable for a model.

Finally, the model is used to make predictions about new inputs.



This can be summarized as a data pipeline.

This pipeline gives us a more accurate picture of machine learning than the simplified expression:

Data → Model → Prediction

There are several stages between the world and the model.

At every stage, assumptions are introduced.

The collection process determines what is observed.

The dataset determines what is available.

Preparation determines how the information is transformed.

Representation determines how the model receives it.

Training determines what relationships are learned.

Evaluation determines how those relationships are judged.

The final prediction therefore depends on a chain of decisions rather than on the learning algorithm alone.

The Hidden Structure of a Dataset

A dataset may look objective because it is represented as rows and columns.

But every dataset has a history.

Someone decided what to measure.

Someone selected the sources.

Someone defined the variables.

Someone designed the collection process.

Someone determined which observations would be included.

Someone created or assigned labels.

Someone decided how missing values would be handled.

These decisions become part of the resulting dataset, even if they are not visible in the final file.

This is why experienced data scientists do not begin with the assumption that a dataset is simply "there."

They investigate its origin.

A useful dataset description should ideally answer questions such as:

Where did the data come from?

When was it collected?

How was it measured?

What population does it represent?

What variables were recorded?

How were labels produced?

What observations were excluded?

What limitations are known?

These questions are part of understanding the dataset itself.

In other words, data analysis begins before the first model is trained.

A Complete Example: Learning From Images

Consider a simple classification problem in which the goal is to distinguish between two categories of objects.

The first step is data collection.

Images must be obtained from some source.

The second step is representation.

Each image must be encoded in a computational form, such as an array of pixel values.

The third step is labeling.

Each image needs a target category if the task is supervised learning.

The fourth step is data preparation.

Images may need to be resized or transformed into a consistent format.

The fifth step is training.

A machine learning algorithm receives examples and attempts to learn statistical relationships between the input representation and the target.

The sixth step is evaluation.

The model is tested on data that was not used to fit its parameters.

At each stage, a different question appears.

Are the images relevant?

Are the labels reliable?

Are the training and evaluation sets separated correctly?

Does the representation preserve useful information?

Could information from the evaluation set have leaked into training?

The important lesson is that the final model is only one component of the system.

A sophisticated algorithm cannot automatically compensate for every weakness in the data.

The Big Idea

Data is the bridge between the world and computation.

The physical world contains objects, events, measurements, behaviors, and processes. Data provides a representation of selected aspects of that world in a form that can be stored and processed.

In machine learning, observations are often organized into datasets. Their measurable properties become features, and in supervised learning, known target values become labels.

But representation alone is not enough.

We must also consider whether the data is accurate, complete, consistent, relevant, timely, and representative of the environment in which the system will operate.

We must understand missing data and noise.

We must examine how samples were selected.

We must consider how labels were produced.

We must look for systematic sources of bias.

And we must prevent information that would not be available at prediction time from leaking into training or evaluation.

These are not secondary concerns.

They determine what information the learning algorithm has access to in the first place.

A model cannot learn directly from the world.

It learns from the representation supplied to it.

That gives us the central idea of this chapter:

Before asking how a machine learns, we must understand what the machine is being allowed to learn from.

The next chapter takes the next step.

We have data.

We have observations.

We have representations.

Now we need to understand the learning process itself.

Next Chapter 03

How Machines Learn

A dataset contains observations, but observations alone do not produce a model.

Some procedure must extract relationships from those observations.

The machine must determine which patterns are useful, which relationships can generalize to new cases, and how an input can be transformed into a prediction.

Chapter 03

How Machines Learn

Machine learning is one of the most important concepts in modern artificial intelligence. The idea of machine learning is to design systems, that are able to improve their behavior through learning instead of being programmed to behave in a specific way. The concept itself is a simple one, however, when we try to put it into mathematical formulations, the questions arise; what exactly does it mean for the machine to learn?

In general, machine learning systems learn patterns from data to be able to make a prediction or perform some tasks in a way, that would have been difficult to program explicitly. One needs to understand the following; when using machine learning algorithms, we do not provide them with a set of rules to follow, but instead, we give them some examples to help the algorithm figure out the rules itself.

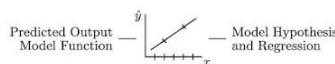
There is a major difference between the two approaches. We will discuss it in more detail later, however, for now, let us note it as such; in programming, we define the rules for transforming input into output. In machine learning, we describe the desired behavior and let the algorithm find the transformation rules.

To put it another way, a program can have a rule, that looks like this:

output = input1 + input2

Meanwhile, a machine learning model can be fed with multiple examples of inputs and outputs and will attempt to find a mathematical function, that transforms the input into the output:

$$\hat{y} = f(x)$$



The purpose of this chapter is to build upon that idea. We will discuss what exactly a machine learning model is, how they are trained, what the major classes of machine learning algorithms exist, and why a machine learning model needs to learn patterns, that it did not see during training.

The Idea of Machine Learning

From Rules to Relationships

Let us now formalize the idea a bit. Let us say, that we have the following problem; given the size of a house, predict its price. Suppose, that we have a set of examples of size-price pairs. It is obvious to us, that the larger the house is, the higher is its price. However, how can we teach our model to find that connection? To do this, we can use a linear function to describe the relationship between size and price.

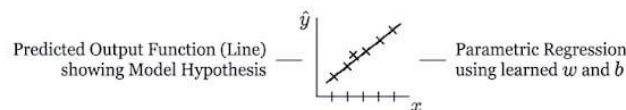
Let:

x be the size of the house

y be the price of the house

Therefore, the relationship between **x** and **y** can be described as:

$$\hat{y} = wx + b$$



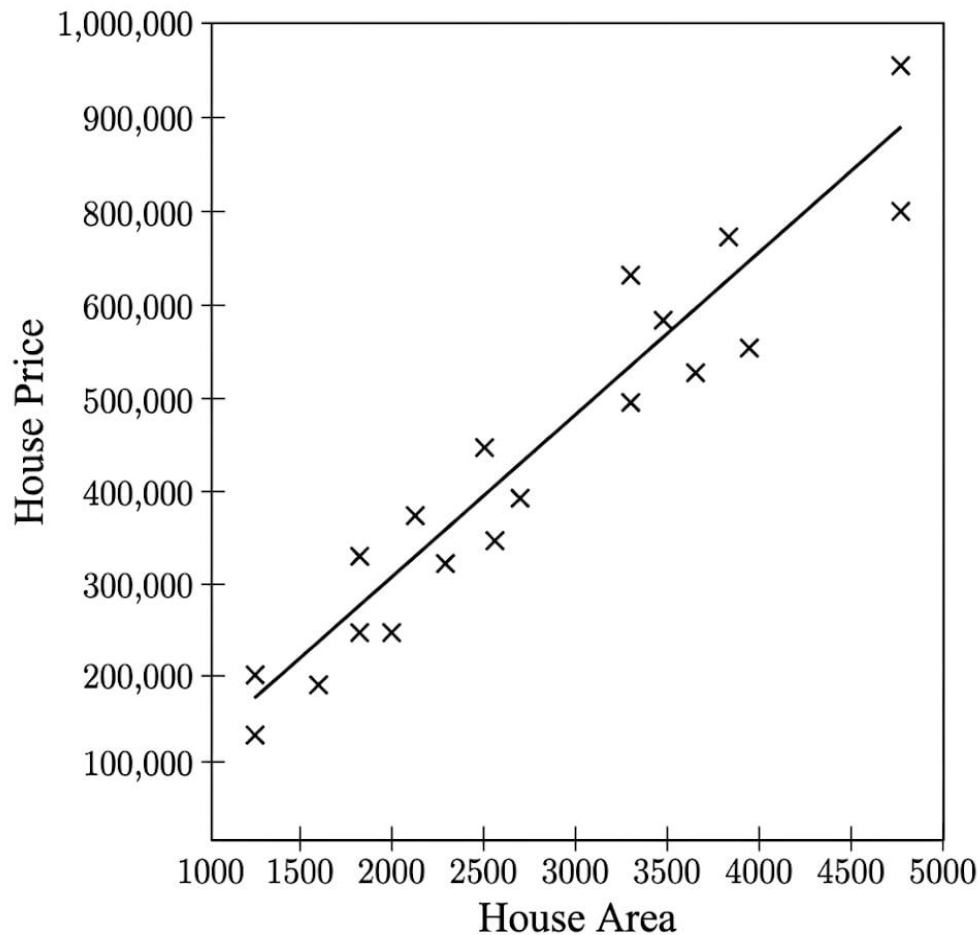
$\hat{y} = wx + b$ - Hypothesis, where 'w' is weight, 'b' is bias, and ' $\hat{y}$ ' is prediction.

Linear Model with Parameterized Hypothesis

Where **w** and **b** are coefficients, that we will determine during the training process.

Given, that we have a set of examples, we can find the values of $\mathbf{w}$ and $\mathbf{b}$, that will allow us to approximate the relationship between x and y most closely. Here is how our examples can look like:

$(\mathbf{x}_1, \mathbf{y}_1)$, $(\mathbf{x}_2, \mathbf{y}_2)$, $(\mathbf{x}_3, \mathbf{y}_3)$, and so on.



The graph illustrates an important idea. The individual points are observations, while the line is a model of the relationship contained in those observations. The model does not reproduce every point exactly. Instead, it attempts to capture the general relationship.

This distinction becomes increasingly important as machine learning problems become more complicated. A useful model is not necessarily one that reproduces every training example perfectly. It is one that captures relationships that remain useful when new data is presented.

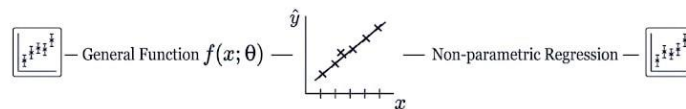
A Model as a Function

We could define our model as a function mapping an input to an output: $\mathbf{y}^{\wedge} = \mathbf{f}(\mathbf{x}; \boldsymbol{\theta})$

Theta here are the parameters of the model: things that define how it operates. In our linear example above, theta would be the vector containing $[w, b]$.

The parameters are defined such that when we apply them, via this function, to an observation, we get a prediction. For a given observation we can write:

$$\hat{y} = f(x; \theta)$$



$\hat{y} = f(x; \theta)$ – Model, where ' θ ' is the vector of parameters, ' x ' is the input, ' f ' is the function, and ' $\hat{y}$ ' is the prediction.

Generalized Model with Non-parametric or High-order θ

The actual value is y , so we can see how far off our prediction was. This is a measure of the error for the observation.

Learning from Examples

Training Data

The examples provided to a learning algorithm are often referred to as the training data. In a supervised learning task like the one we're doing here, each example will be an input-output pair.

We could represent the data as:

$$\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

$$\mathcal{D} = \{(x_i, y_i)\}$$

The x's are our inputs, and the y's are our targets: the values that we want our model to predict.

We apply our model to the x's to get our prediction, and the algorithm compares it to the real value to see how far off it was. For example, in the case where the real price of a house is:

$$y = 2,000,000$$

but our model predicts:

$$y^{\wedge} = 1,850,000$$

Then we can say that our prediction had an error of:

$$\text{error} = |y - y^{\wedge}|$$

In this case, the error would be:

$$\text{error} = |2,000,000 - 1,850,000|$$

$$\text{error} = 150,000$$

But how do we measure the overall performance of the model? A single number like this isn't going to tell us much about how well our model actually performs across all of our data. That's why machine learning algorithms tend to use some sort of loss or cost function to aggregate all of these errors into a single number that can be optimized.

Take the Mean Squared Error, for instance. We could represent that in plain text as:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

What this means is, for every observation, we're going to take the difference between the actual value and the predicted value, square that difference, add up all of those, and then divide by the total number of observations.

We square the difference to penalize larger errors more heavily, which has some useful properties (don't worry about this for now).

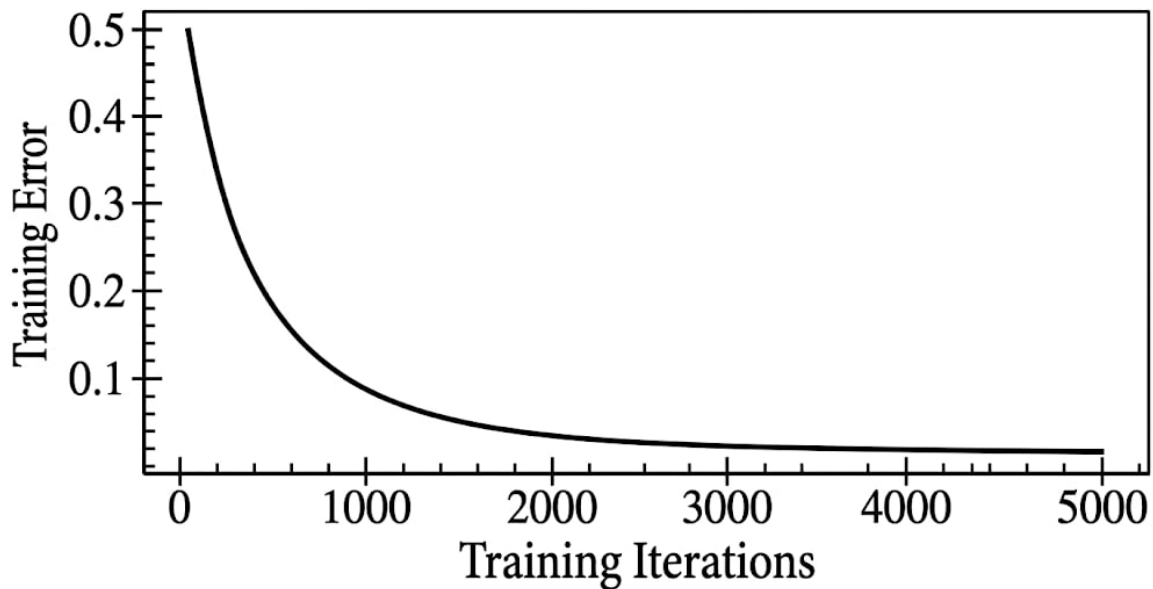
So, once we've defined a way to measure the error, we can define learning as nothing more than the optimization of this loss function.

$$\theta^* = \underset{\theta}{\operatorname{argmin}} L(\theta)$$

In this case, theta is the set of parameters found by minimizing the loss function L of theta. What that means in practical terms is that we use an iterative algorithm to try different values of theta until we find the ones that give us the best loss value. The algorithm doesn't know what the right values are ahead of time, so it tries combinations and measures the loss until it finds something that works.

This trial and error are what we refer to as training.

So, as you can see, building a machine learning model isn't quite as simple as "feeding data to a computer". We have to define the model itself, measure the error, and optimize the parameters based on that error.



The graph represents the general behavior of an optimization process. During training, the model parameters are repeatedly adjusted. As the parameters become more suitable for the training data, the measured training error may decrease.

However, decreasing training error is not the final objective of machine learning.

A model can become extremely good at reproducing the training examples while becoming less useful on new examples. This is one of the most important problems in machine learning, and it leads to the idea of generalization, which we will return to later in the chapter.

Learning Is Not Memorization

A machine learning model is expected to do more than remember the examples in its training dataset.

Suppose a model is trained to recognize handwritten digits. The training dataset may contain thousands of images of the digits 0 through 9. The model will not encounter exactly the same image every time it is used. Instead, it must recognize new handwriting produced by different people, under different conditions.

The useful information is therefore not the individual image itself. It is the structure shared by many examples.

Learning occurs when a model captures patterns that allow it to make useful predictions beyond the observations used during training.

Supervised Learning

In supervised learning, the training data contains both inputs and known target outputs.

The model receives examples of the form:

(x_1, y_1) , (x_2, y_2) , and

and learns a relationship between x and y .

Two major supervised learning tasks are classification and regression.

In classification, the output belongs to a discrete category.

For example, an email classification model might assign one of two labels:

spam

not spam

The model learns from labeled examples and attempts to assign the correct category to new inputs.

In regression, the output is a numerical quantity.

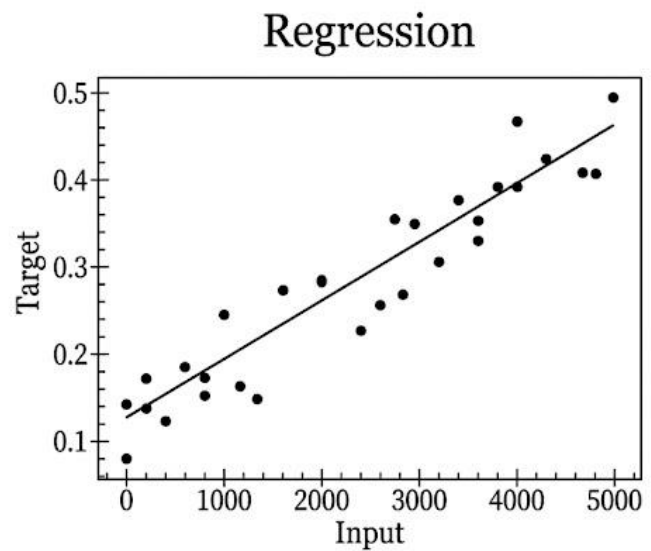
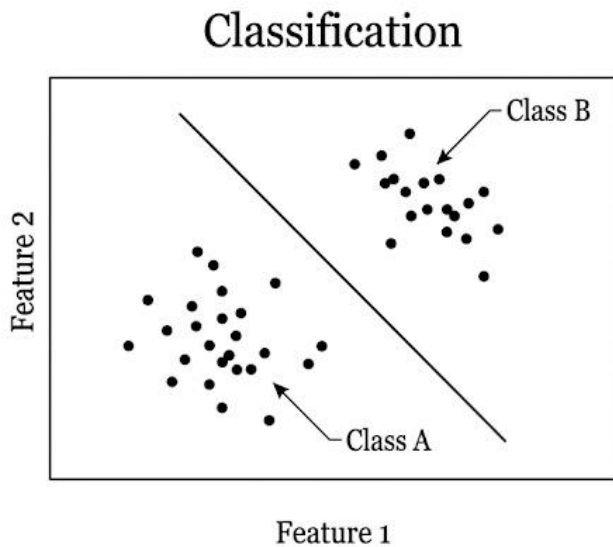
Examples include predicting temperature, estimating house prices, or predicting the amount of energy produced by a system.

The distinction can be expressed simply:

Classification -> discrete output

Regression -> numerical output

The input may be complex in either case. An image, for example, can be used as input to a classification model that predicts which category the image belongs to.



The difference between the two tasks is therefore not simply the algorithm being used. It is also the nature of the quantity being predicted. Classification produces a category, while regression estimates a numerical value.

Unsupervised Learning

In unsupervised learning, the training data does not provide a target value for each example.

Instead of learning:

input -> known answer

the algorithm searches for structure within the inputs themselves.

One common example is clustering. Suppose a dataset contains observations described by several numerical features, but no category labels are provided. A clustering algorithm may identify groups of observations that are similar to one another.

The resulting groups are not necessarily "correct answers" supplied by a teacher. They are structures discovered from the relationships among the observations.

This makes unsupervised learning useful for exploratory analysis and for situations in which labeled data is limited or unavailable.

Reinforcement Learning

Reinforcement learning uses a different form of feedback. An agent interacts with an environment, takes actions, and receives rewards or penalties based on the consequences of those actions.

The goal is not simply to reproduce a set of labeled examples. Instead, the agent must learn a strategy that produces useful long-term outcomes.

The general structure can be represented as:

State -> Action -> Environment -> Reward -> New State

This setting is particularly important when a sequence of decisions matters. The quality of an action may depend not only on its immediate result but also on what happens later.

These three settings provide a useful foundation for understanding machine learning:

Supervised learning -> learn from labeled examples

Unsupervised learning -> discover structure in data

Reinforcement learning -> learn from interaction and feedback

The categories are conceptually different, although modern systems can combine ideas from more than one learning setting.

Learning Through Patterns

The central challenge of machine learning is not obtaining a low error on the training data. It is obtaining useful performance on data that the model has not seen before. This ability is known as generalization.

From Training to Prediction

Consider a model trained using a collection of observations.

During training, the model has access to the training examples. After training, however, it must operate without knowing the correct answer in advance.

The complete process can therefore be viewed as:

Training data -> Learning algorithm -> Model -> New input -> Prediction

The training data determines what the model can learn. The learning algorithm determines how the model searches for a useful relationship. The resulting model is then applied to new inputs.

This distinction between training and prediction is essential.

A model that performs well only on its training data has limited value. A useful model must preserve the relationships it learned when it encounters observations that were not part of the training process.

Generalization

Suppose two models are trained on the same dataset.

Model A produces very small errors on the training examples but performs poorly on new examples.

Model B produces slightly larger training errors but performs consistently on both training and new examples.

The second behavior is generally more desirable for a predictive system because the purpose of learning is to make predictions beyond the training observations.

This leads to the idea of model capacity.

A model with very limited capacity may be unable to represent the underlying relationship in the data. This situation is called underfitting.

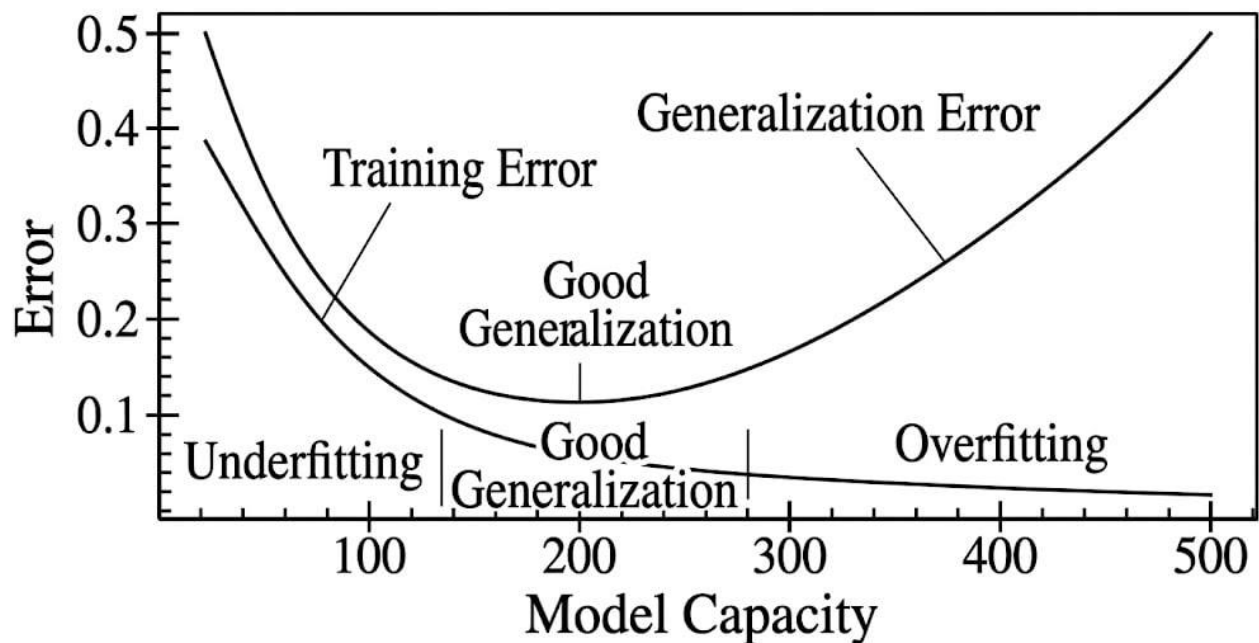
At the opposite extreme, a model with very high capacity may become excessively adapted to the training data. It can begin to reproduce noise or accidental details instead of

learning the underlying structure. This is called overfitting.

The relationship between capacity and performance is therefore not simply:

more capacity = better model

Instead, the model needs an appropriate level of capacity for the problem and the available data.



This relationship explains why training a model is not simply a matter of minimizing training error as much as possible. A model that has memorized details of the training dataset may have very low training error while producing poor predictions on new data.

The difference between these two situations can be described using two types of error:

Training error = error measured on examples used for learning

Test error = error measured on previously unseen examples

The test error is often more informative about how well the model generalizes, provided that the test data is representative of the conditions in which the model will eventually be used.

Learning as Approximation

It is important to understand what the model has actually learned.

A machine learning model does not necessarily discover a perfect description of reality. It constructs an approximation based on the information available in the training data and the assumptions built into the model.

If the data is incomplete, noisy, or biased, the learned model can reflect those limitations. If the model is too simple, it may fail to represent important relationships. If it is too flexible, it may adapt too closely to the training observations.

Learning is therefore a balance between fitting the available data and preserving the ability to generalize.

This perspective also explains why machine learning is closely connected to mathematics, statistics, and optimization. The model represents a mathematical relationship, statistics helps describe uncertainty and variation in data, and optimization provides methods for finding parameters that improve the model.

The Larger Picture

Machine learning can now be understood as a process rather than as a single algorithm.

A dataset provides observations.

A model provides a mathematical structure for representing relationships.

A loss function provides a way to measure error.

An optimization method adjusts the model parameters.

Evaluation determines whether the resulting model can generalize to new data.

These ideas form the foundation for many machine learning systems. Linear regression, decision trees, neural networks, and modern deep learning architectures differ greatly in their internal structures, but they all involve the fundamental problem of learning useful relationships from data.

The next step is to examine the mathematical language that makes these relationships possible.

Machine learning does not exist separately from mathematics. Functions describe models, vectors represent data, matrices organize large collections of values, probability describes uncertainty, and optimization provides methods for improving model parameters.

Understanding these mathematical ideas will make the algorithms that follow much easier to understand.

Next Chapter

Chapter 04: The Mathematics Behind AI

In the next chapter, we will develop the mathematical tools used throughout artificial intelligence. We will begin with functions and representations, then move to vectors and matrices, probability and statistics, and finally gradients and optimization. These concepts will provide the mathematical foundation needed to understand how modern AI models represent information and learn from data.

Chapter 04

The Mathematics Behind AI

Artificial intelligence is constructed with mathematical constructs. The machine learning algorithm does not operate on notions such as "image," "price," or "language" as one would expect humans to use. Mathematical language is required to represent such notions as numeric variables. Moreover, mathematics is the language that describes such variables, the relations between them, and the process of learning them.

The mathematical basis of AI is wide-ranging, although a limited number of mathematical concepts recur in machine learning and deep learning algorithms. Functions capture the relationship between input and output variables. Vectors and matrices structure the numerical data. Probability and statistics offer techniques for dealing with uncertainties and data. Optimization enables the model to fine-tune the parameters.

In this chapter, we discuss some basic mathematical concepts without covering all of mathematics. Our aim is only to set the notation and intuition for the following chapters.

Functions and Representations

A function describes a relationship between quantities. In its simplest form, a function takes an input and produces an output: $y = f(x)$

For example, consider the function: $y = 2x + 1$

If $x = 3$, then:

$$y = 2(3) + 1 = 7$$

In machine learning, the function is usually more important than the particular numerical example. A model attempts to represent a relationship between inputs and outputs.

A simple linear model can be written as:

$$\hat{y} = wx + b$$

where:

$\mathbf{x}$ = input

$\mathbf{y}^\wedge$ = predicted output

$\mathbf{w}$ = parameter controlling the slope

$\mathbf{b}$ = parameter controlling the intercept

The parameters determine the behavior of the model. Changing w changes how strongly the output responds to x , while changing b shifts the function.

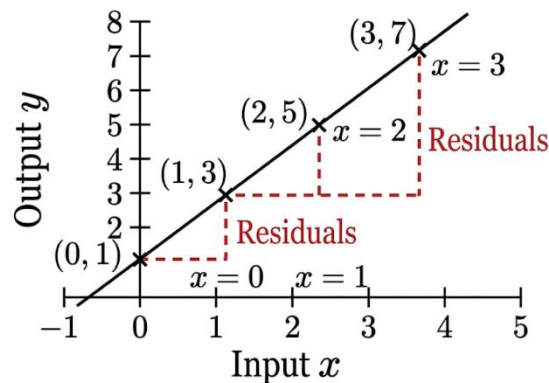
For multiple inputs, the same idea can be extended:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

This form appears throughout machine learning. A prediction may depend on many features, each contributing to the final result.

The mathematical representation of the inputs becomes increasingly important as the number of features grows. This is where vectors and matrices become useful.

$$y = 2x + 1$$



Vectors and Matrices

A vector is an ordered collection of numbers.

A vector can be written in plain text as:

$$\mathbf{x} = [\mathbf{x1}, \mathbf{x2}, ..., \mathbf{xn}]$$

For example:

$$\mathbf{x} = [2, 5, 7]$$

A vector can represent the features of one observation. If a house is described by its area, number of rooms, and age, its feature vector might be represented as:

$$\mathbf{x} = [120, 4, 8]$$

The numbers themselves have meaning because they correspond to specific features.

Vectors also provide a compact way to express mathematical operations. The linear model introduced earlier can be written as:

$$\hat{y} = w^T x + b$$

Here, $\mathbf{w}^T$ is the transpose of the parameter vector w . The expression $\mathbf{w}^T \mathbf{x}$ represents the dot product between the two vectors.

If: $\mathbf{w} = [\mathbf{w1}, \mathbf{w2}, ..., \mathbf{wn}]$

and: $\mathbf{x} = [\mathbf{x1}, \mathbf{x2}, ..., \mathbf{xn}]$

then:

$$w^T x = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$$

This notation becomes extremely useful when a model contains many features.

A matrix is a rectangular arrangement of numbers. For example:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$$

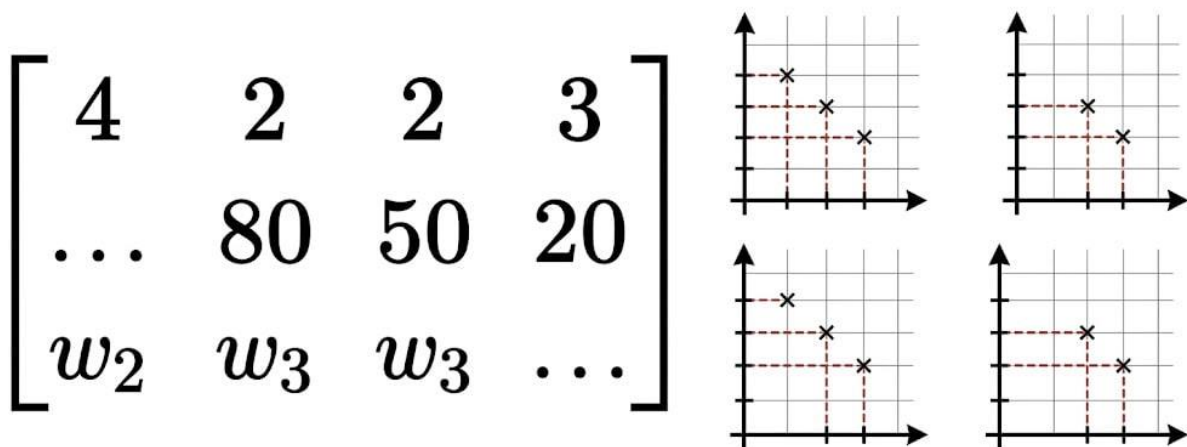
The matrix above contains 2 rows and 3 columns.

Matrices are useful because machine learning datasets naturally form this structure. If each row represents one observation and each column represents one feature, an entire dataset can be represented by a matrix:

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1m} \\ x_{21} & x_{22} & \dots & x_{2m} \\ \dots & \dots & \dots & \dots \\ x_{n1} & x_{n2} & \dots & x_{nm} \end{bmatrix}$$

Here, n represents the number of observations and m represents the number of features.

This representation allows mathematical operations to be applied to large datasets efficiently. Modern machine learning systems rely heavily on vector and matrix operations, especially neural networks.



Probability and Statistics

Data is rarely perfectly predictable. Measurements contain variation, observations may be incomplete, and different inputs can produce different outcomes. Probability provides a mathematical framework for describing uncertainty.

A probability represents how likely an event is to occur.

For an event A: $0 \leq P(A) \leq 1$

A probability of 0 represents an impossible event, while a probability of 1 represents an event that is certain under the defined conditions.

For example, a classification model may produce probabilities for several possible classes:

$P(\text{Class A} \mid \mathbf{x}) = 0.70$

$P(\text{Class B} \mid \mathbf{x}) = 0.20$

$P(\text{Class C} \mid \mathbf{x}) = 0.10$

The notation:

$P(A \mid B)$

is read as "the probability of **A** given **B**."

In machine learning, this conditional probability is especially important because predictions often depend on observed evidence. A model may estimate the probability of a class given an input:

$P(\mathbf{y} \mid \mathbf{x})$

This provides a more informative description than simply producing a category. Instead of saying that an input belongs to Class A, a probabilistic model may estimate how strongly the available evidence supports that classification.

Statistics provides another essential tool: describing the behavior of data.

The mean of n values is: **$\text{mean} = (\mathbf{x1} + \mathbf{x2} + \dots + \mathbf{xn}) / n$**

or, using summation notation:

$$\text{mean} = \frac{1}{n} \sum_{i=1}^n (x_i)$$

The mean provides a measure of the central tendency of a dataset.

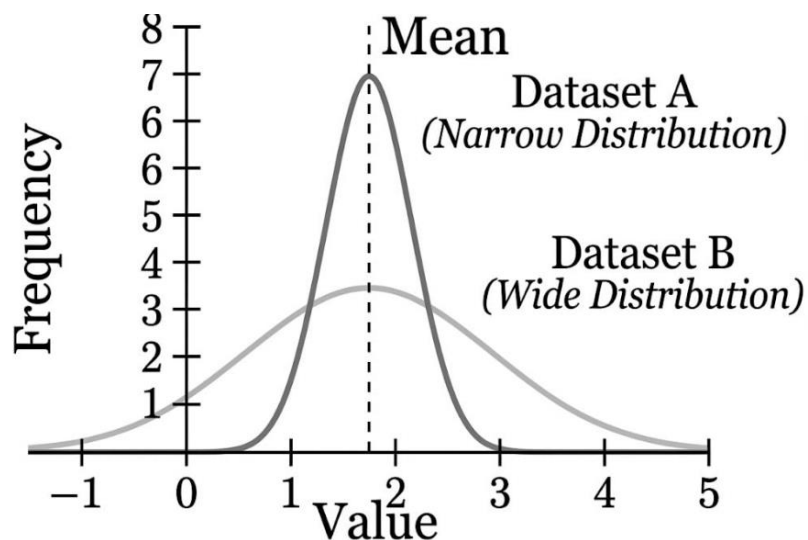
The spread of the observations can be described using variance:

$$\text{variance} = \frac{1}{n} \sum_{i=1}^n (x_i - \text{mean})^2$$

A larger variance indicates that the observations are more spread out around the mean.

These concepts become important when analyzing datasets, comparing measurements, and understanding how much variation a model must handle.

Probability and statistics also help explain why a model's prediction should not always be treated as absolute certainty. A model operates using information from data, and that information can contain uncertainty.



Gradients and Optimization

Machine learning models contain parameters that must be adjusted during training. The central question is how the algorithm should change those parameters to reduce error.

Suppose a model has a loss function:

$L(w)$

where w represents a model parameter.

If changing w changes the loss, then the derivative tells us the direction in which the loss changes:

$$\frac{dL}{dw}$$

If:

$$\frac{dL}{dw} > 0$$

then increasing w locally increases the loss.

If:

$$\frac{dL}{dw} < 0$$

then increasing w locally decreases the loss.

For models with many parameters, we use the gradient.

If:

$$\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{bmatrix}$$

then the gradient of the loss is:

$$\nabla_{\theta} L(\theta) = \begin{bmatrix} \frac{\partial L}{\partial \theta_1} \\ \frac{\partial L}{\partial \theta_2} \\ \vdots \\ \frac{\partial L}{\partial \theta_n} \end{bmatrix}$$

The gradient points in the direction of greatest local increase in the loss. Therefore, to reduce the loss, an optimization algorithm can move in the opposite direction.

This produces the basic gradient descent update:

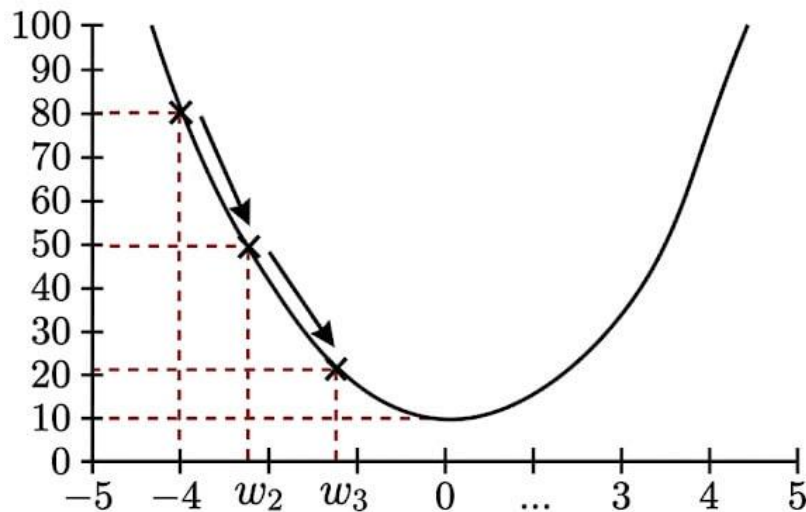
$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_{\theta} L(\theta_{\text{old}})$$

where: Θ = model parameters

η = learning rate

$\nabla \theta L(\theta)$ = gradient of the loss

The learning rate controls the size of each update. If it is too small, training may progress slowly. If it is too large, the optimization process may move too far and fail to approach a useful minimum.



The figure provides a geometric interpretation of optimization. The model begins at some parameter value, evaluates the loss, calculates the direction in which the loss increases, and moves in the opposite direction.

Repeated updates can gradually move the parameters toward a region where the loss is smaller.

The complete learning process can therefore be summarized mathematically as:

Data -> Model -> Prediction -> Loss -> Gradient -> Parameter Update

The same basic structure will appear again when we study neural networks. A neural network may contain thousands, millions, or billions of parameters, but the underlying principle remains the same: parameters are adjusted according to information provided by the loss.

The Mathematical Foundation

The mathematics introduced in this chapter forms a small but important foundation for artificial intelligence.

Functions describe how inputs are transformed into outputs.

Vectors provide a compact representation of features and parameters.

Matrices organize large collections of numerical information.

Probability provides a language for uncertainty.

Statistics describes patterns and variation within data.

Gradients indicate how a function changes with respect to its parameters.

Optimization provides methods for finding parameter values that improve a model.

These ideas are closely connected. A machine learning model can be represented as a function, its inputs can be stored as vectors, large datasets can be represented as matrices, uncertainty can be expressed probabilistically, and optimization can adjust the model parameters.

The mathematics itself is not the purpose of artificial intelligence. It is the language that allows an AI system to represent information and learn relationships precisely.

In the next chapter, these ideas will be combined into a complete machine learning workflow, from defining a problem and preparing data to training a model and evaluating its performance.

Next Chapter

Chapter **05**:

Building a Machine Learning Model

Building a machine learning model involves more than choosing an algorithm and training it. The problem must first be defined, the data must be prepared and represented correctly, and the model must then be trained and evaluated. The next chapter brings these steps together and follows the process of building a machine learning model from the definition of the problem to the evaluation of its performance.

Chapter 05

Building a Machine Learning Model

Building a machine learning model does not consist only in feeding some data into an algorithm. The construction of a good model requires a series of choices: problem definition, data preparation, choice of a suitable model, its training, and testing of its performance on unseen data.

This sequence of steps relates the concepts discussed in the previous chapters. The data gives the information, mathematics gives the representation, and machine learning gives the way to discover a useful relationship in this information.

Defining the Problem

The initial stage of a machine learning project is the definition of what is to be predicted by the system.

A problem can always be stated using three main elements:

Input -> Model -> Output

The input comprises the data that is known to the model, while the output is the value or class which the model needs to produce.

In the case of supervised learning, the set of data may be written as:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

where x_i corresponds to the input, while y_i stands for its target.

It depends on the type of the target whether the problem will be solved as a regression or classification.

The proper problem statement also sets the requirement on the data and model evaluation metrics.

Preparing the Data

Since a model is only able to learn from the data that is available to it, it is necessary to understand and prepare the dataset before training begins.

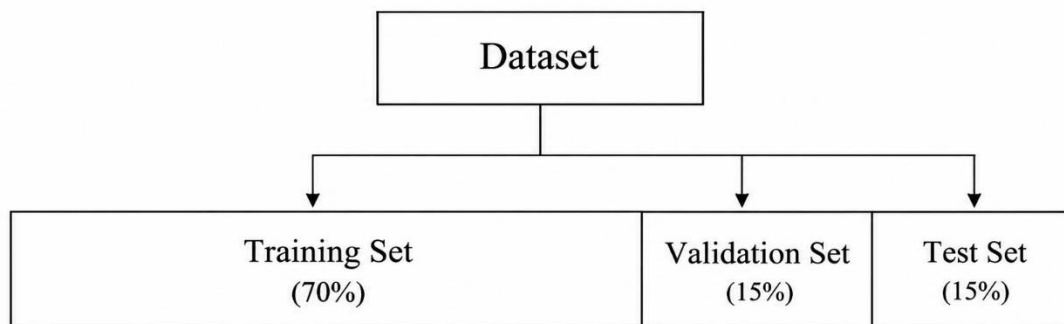
The data can be split into different sets as follows:

Training Set --> used to train the model parameters

Validation Set --> used to make decisions about the model

Test Set --> used for testing the performance of the model

It is critical to do so since performance estimation using the same observations that have been used for training will not give an accurate indication of the model's performance on unseen data. This separation has gained much attention in recent machine learning literature.



Data preparation may also involve handling missing values, transforming numerical features, encoding categorical information, or scaling values when required by the chosen algorithm.

The goal is not to transform the data unnecessarily. The goal is to provide the model with a representation that is suitable for learning.

One important principle is to prevent **data leakage**. Information from the validation or test data should not influence the training process or decisions about the model. Otherwise, the measured performance may be artificially optimistic.

Model Training

Now that the problem has been identified and the data has been prepared, we can start training a model.

A model has parameters that govern its behavior. The training process involves finding a set of values of these parameters that minimize the loss on the training data.

It can be thought of as an Input **Data -> Model -> Prediction -> Loss -> Parameters Update process.**

Mathematically, the goal can be expressed as:

$$\theta^* = \underset{\theta}{\operatorname{argmin}} L(\theta)$$

where θ is the set of model parameters and $L(\theta)$ is the loss function.

The optimization techniques described in chapter 04 are used for updating the value of θ .

However, training alone does not give us information about the usefulness of a model. A model can have extremely low training loss by learning the training data very well.

It brings up one of the main differences between optimization and generalization.

Optimization means improving performance on the training data.

Generalization means maintaining performance on unseen data.

Both are needed in machine learning.

Hyperparameters

Not all variables in a machine learning system are learned from data.

Parameters are learned from data.

Hyperparameters are specified either before or during the training. Learning rate, model complexity, and some forms of regularization are examples of hyperparameters.

The key difference can be summarized as follows:

Parameters -> learnable from data

Hyperparameters -> specified to control learning process

Hyperparameters can be fine-tuned using validation data rather than the test data itself. It helps avoid including the test data into the model selection process.

Evaluating the Model

Once trained, the model has to be evaluated using the data which wasn't used to estimate its parameters.

The evaluation measure will vary depending on the task.

For regression task one popular measure is Mean Squared Error:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

For classification, accuracy measure can be formulated as follows:

Accuracy = **(Number of correct predictions) / (Total number of predictions)**

These measures allow quantifying the model performance.

However, a single value must never be looked at independently of the dataset and the goal that the model needs to achieve. A model can have high accuracy but still perform badly for a minority class.

It's for this reason that evaluation can't just be thought of as being the last step of a machine learning process.

Generalization and Model Misbehavior

The most crucial criterion for testing a model is its performance on unseen data.

Let us assume that the training error continues to decline, but the validation error starts to rise. The model becomes increasingly dependent on the training data.

It is said to overfit the data.

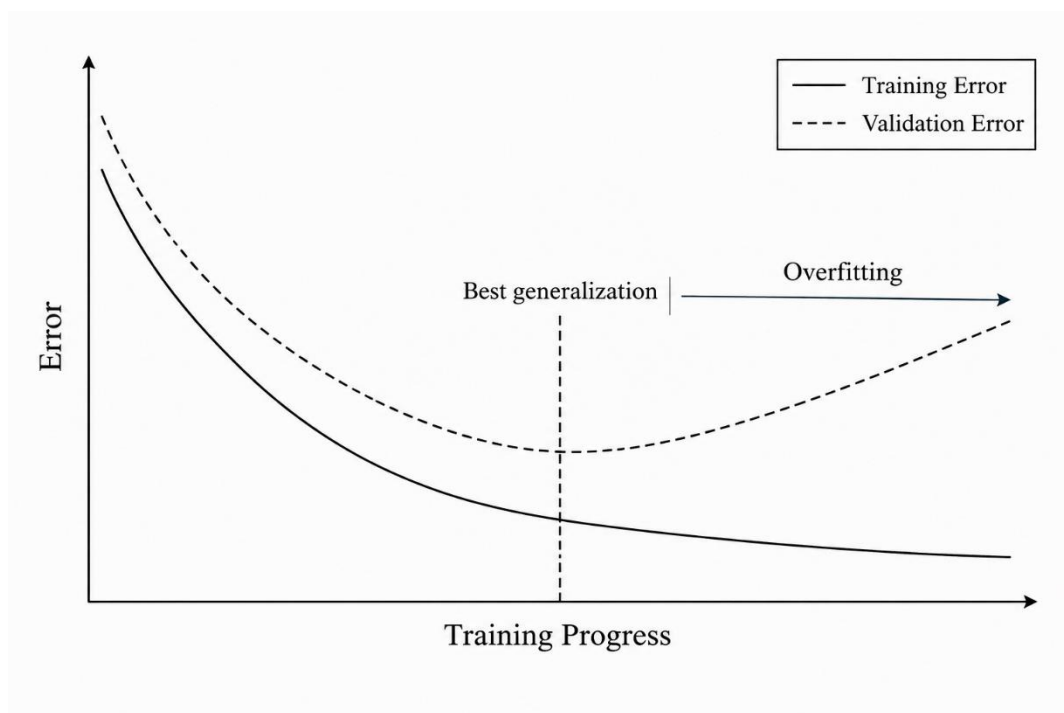
In case the model is too simplistic to be able to find any meaningful pattern in the data, then it will fail to work on both the training data and new data. It is underfitting.

The connection can be expressed as follows:

Underfitting: Model is too narrow

Good generalization: Meaningful patterns found

Overfitting: Training data-dependent patterns found



Making a Model Better

However, when a model fails to perform well, it might not necessarily require additional complexity.

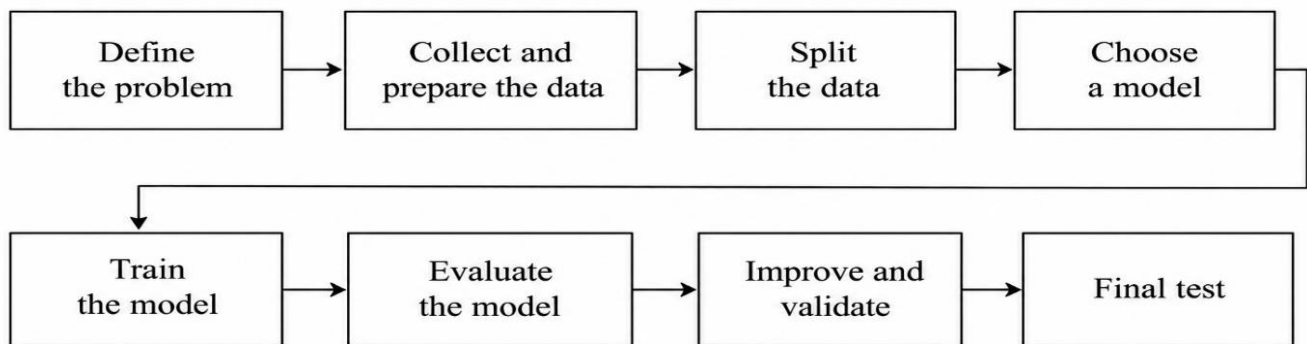
In other words, it must first be determined if the available data properly reflects the problem. If the data is inadequate, then no matter how sophisticated an algorithm is used, the quality of the model will suffer.

Otherwise, one might improve the model by modifying the model itself, tuning its parameters, improving the input representation, and preventing overfitting.

One technique used to prevent a model from overfitting is called regularization; here, the process of learning is somehow constrained, so that the model will not become overly dependent on unnecessary complexity.

Other techniques include improvements in the dataset itself, feature selection, and terminating learning before the overfitting occurs.

These are some of the concepts repeated throughout modern discussions of machine



The process is iterative rather than strictly linear. Evaluation may reveal problems in the data or model, requiring earlier decisions to be reconsidered.

The central idea is simple: building a machine learning model is not only about finding a function that fits existing data. It is about constructing a system that can extract useful patterns from that data and apply them to new situations.

This distinction between fitting and generalization will become even more important when we move from traditional machine learning models to neural networks.

Next Chapter

Chapter 06: The Artificial Brain

Neural networks provide a different way to represent complex relationships. Instead of manually defining many rules, they use connected computational units whose parameters can be learned from data. The next chapter introduces the artificial neuron, neural network architecture, and the basic mechanisms that allow these networks to learn.

Chapter 06

The Artificial Brain

The machine learning algorithms can find useful relations in datasets; however, there are tasks which need a model able to learn patterns on different hierarchical levels of complexity. The concept of the neural network helps to perform this by building many simple computational elements into complex structures of layers.

It is true that the concept of the neural network was inspired by the connectivity of neurons of living beings. However, the artificial neural network is actually just a mathematical model and not a replication of the human brain. In particular, it uses mathematical operations in order to map numeric inputs to outputs, tuning its parameters during learning.

Neural network takes inputs, applies multiple transformations and returns a prediction. During training phase, the parameters of these transformations are tuned in such a way that the prediction gets closer to the target value. With appropriate depth and learning technique, neural network can learn very complex patterns hard to define in any other way.

In this chapter, the reader is introduced to the concepts of the artificial neuron, the building of neural networks, loss function and learning via it and backpropagation algorithm in them. The notion of deep learning is explained.

The Artificial Neuron

From Biological Neurons to Artificial Neurons

Neurons in biology receive input from other neurons, process the input, and can pass on a signal to other neurons. The very notion of receiving inputs, processing them, and producing outputs has been used to create the mathematical construct of the artificial neuron.

An artificial neuron is quite different from a biological neuron. It takes in numeric inputs,

gives weights to those inputs, aggregates them, applies a bias, and finally an activation function.

It does not matter for the artificial neuron to be biologically accurate. The significance of the artificial neuron lies in the fact that many such neurons can be aggregated to learn complex mathematical functions.

Inputs and Weights

Suppose a neuron receives several inputs:

$x_1, x_2, \dots, x_n$

Each input has an associated weight:

$w_1, w_2, \dots, w_n$

The weights determine how strongly the inputs affect the neuron's output. If a weight is large and positive, its corresponding input has a stronger positive contribution. A negative weight can reduce the contribution of an input.

The neuron first calculates a weighted sum:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

The value **b** is called the bias. The bias allows the neuron to shift the result of the weighted sum and gives the model additional flexibility.

The weights and bias are not normally chosen manually. They are parameters learned during training.

Activation Functions

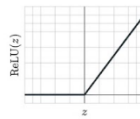
The weighted sum passes through an activation function:

$$a = f(z)$$

An activation function dictates how the neuron manipulates the weighted sum before producing the output.

One commonly applied activation function is called the Rectified Linear Unit, or **ReLU**:

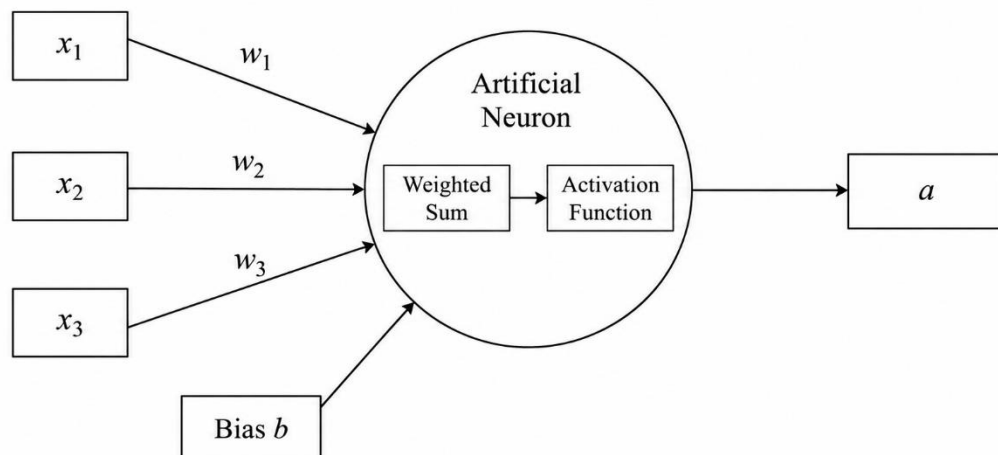
$$\text{ReLU}(z) = \max(0, z)$$



For $z > 0$, the **ReLU** gives back z . For $z < 0$, it produces zero.

It is crucial to have activation functions because they provide nonlinearity to the model. If there were no nonlinear activation functions, the combination of multiple layers would still be close to a linear transformation. With nonlinearity, neural networks can learn representations which cannot be expressed in one linear formula.

Other types of activation functions include sigmoids and hyperbolic tangents.



Building Neural Networks

Layers and Connections

Any single neuron can compute only certain transformations. Neural networks increase their capabilities by linking multiple neurons to form layers.

A simple feedforward neural network is comprised of an input layer, several hidden layers, and an output layer.

The input layer accepts the features of a data sample. Intermediate transformations are computed in the hidden layers, and predictions are generated at the output layer.

Thus, the architecture can be depicted as follows:

Input → Hidden Layers → Output

All connections between neurons have certain weights, and all neurons may also have biases. Consequently, a neural network consists of a large number of trainable parameters.

The overall configuration of neurons and connections is known as the architecture of the network. There are various types of architectures, which are used to solve different problems, but the main idea is still the same – information is being transformed in a series of computational layers.

Forward Propagation

Once an input is given to a neural network, the information moves through the network from the input layer toward the output layer. This process is called forward propagation.

For a single layer, the computation can be written as:

$$z = Wx + b$$
$$a = f(z)$$

Here, **W** represents the weights, **b** represents the bias, and **f** represents the activation function.

The output of one layer becomes the input to the next layer. The network therefore performs a sequence of transformations until it reaches the output layer.

For example:

Input → Layer 1 → Layer 2 → Layer 3 → Prediction

At this stage, the network is not learning. It is simply using its current parameters to produce a prediction. Learning occurs when this prediction is compared with the target and the parameters are subsequently updated.

Hidden Layers

With hidden layers, the network is capable of forming representations of the inputs within itself. It should be noted that such representations are not specified explicitly within the input data. In contrast, they are learned.

Thus, a single layer could form a relatively primitive representation or a pattern of the input. Another layer could then use this representation to form a more complex one.

For instance, while processing an image, primitive layers would detect such features as changes in luminosity or edges. More advanced layers would combine these primitives into something more meaningful.

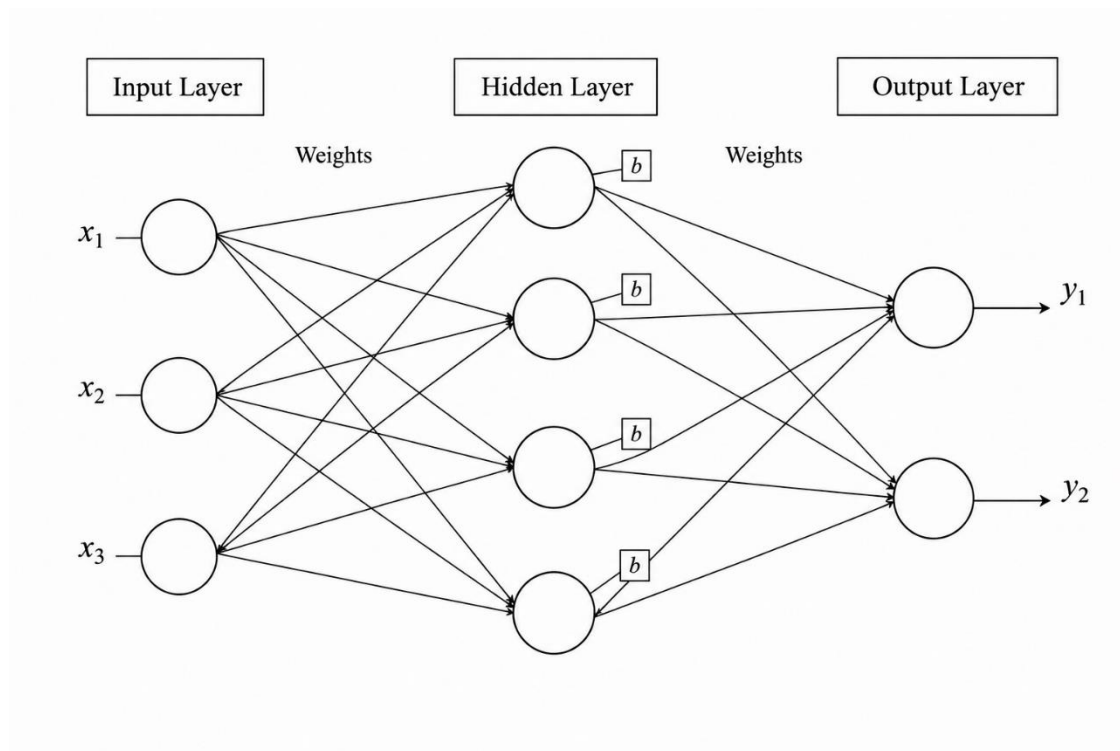
It should be noted that what particular representations a given neural network learns depends on various factors.

Representing Complex Patterns

The power of the neural network is the result of the relationship of many neurons and layers together. A single neuron can conduct only a simple transformation, but the neural network can unite thousands and even millions of these simple transformations.

This enables neural networks to represent a very complex function.

Nevertheless, having more parameters doesn't mean that we will get a better model. The neural network should discover the relations instead of simply memorizing the training set.



Learning in Neural Networks

Loss and Error

Before training begins, the parameters of a neural network are usually initialized with values that do not yet produce accurate predictions.

For a training example, the network produces a prediction:

$$\hat{y} = f(x; \theta)$$

where θ is the set of all parameters of the network.

The prediction is then compared to the target y using the Loss Function as follows

$$\text{Loss} = L(y, \hat{y})$$

The loss measures numerically the difference between the prediction and the actual output.

The training process tries to determine the values of the parameters that minimize this loss, and therefore training a neural network becomes an optimization problem.

Gradient Descent

Parameters of a neural network can be viewed collectively as θ . So, the task of training the model can be defined as:

$$\theta^* = \underset{\theta}{\operatorname{argmin}} L(\theta)$$

One popular technique for optimization is gradient descent.

The gradient includes the partial derivatives of the loss with respect to the parameters:

$$\operatorname{grad} L(\theta) = \left[\frac{dL}{d\theta_1}, \frac{dL}{d\theta_2}, \dots, \frac{dL}{d\theta_{tan}} \right]$$

This tells us how the loss would change with change in parameters.

The parameters are updated accordingly:

$$\theta_{new} = \theta_{old} - \eta \cdot \operatorname{grad} L(\theta_{old})$$

The number eta is known as the learning rate. It determines the size of the update.

If the learning rate is very large, the training process could miss out on good parameter values or even diverge. In case the learning rate is very small, learning would take place much more slowly than required. Proper selection of the learning rate is thus an essential aspect of training a neural network.

Backpropagation

A neural network can have multiple layers with a very high number of parameters. If we want to optimize our model, we need to figure out the contribution of each of these parameters to the final loss.

Backpropagation offers us an efficient way to do this.

Backpropagation starts with a forward propagation step through the whole model. We get a prediction, calculate a loss, and move backwards through the model applying the chain rule to find out how a change of each parameter affects the final loss.

Thus, our full training procedure can be represented as follows:

Input → Forward Propagation → Prediction → Loss

Loss → Backpropagation → Gradients → Parameter Update

The obtained gradients are used by an optimization algorithm to update the parameters of the model.

We repeat this procedure for numerous training samples and iterations. Over time, the parameters can converge to their optimal values making the predictions of our model more accurate.

Updating Parameters

A general training cycle can be broken down into six stages:

1. The network gets the training data.
2. Forward propagation gives a prediction.
3. The loss function computes the error.
4. The backpropagation computes the gradients.
5. The optimizer adjusts the parameters.
6. Repeat the process.

The network doesn't learn by memorizing all training examples. The point of training is to change the parameters of the network such that it can extract useful relationships from the data.

It is an important difference because a good neural network shouldn't just remember the examples it had seen. It should know how to make useful predictions about unseen data using what it has learned.

Deep Learning

Depth and Representation

When a neural network has more than one hidden layer, it is referred to as a deep neural network. Here, "depth" implies the number of layers that are used in the computation of information.

Depth makes it possible for the system to develop representations progressively. Instead of mapping from input to the output in one shot, the system can develop intermediate representations.

A simple understanding of this is:

Raw Input → Simple Representations → More Complex Representations → Prediction

At each step, the representation is transformed by the next layer.

Learning Features Automatically

Learning good representations directly from the data is one of the main benefits of deep learning.

In some conventional machine learning models, humans have to design features in advance of training the model. Deep neural networks, on the other hand, may be able to learn some representations during the training phase.

If the data is an image, then the network can learn representations concerning the visual structures. In case of language data, the network can learn representations regarding the relationships between words or tokens.

This does not mean that the involvement of humans is not needed anymore.

Preprocessing of the data, architecture of the model, training objectives, evaluating the results, and many other aspects need to be thought about thoroughly.

What is different here is that the network may be able to learn a lot of the representation on its own.

Why Deeper Architectures Can Learn Complex Structures

Depth enables a network to perform a series of transformations. The additional depth of a network allows the architecture to encode patterns that could be hard to recognize for a shallower one.

Nevertheless, depth has its pitfalls. Deeper architectures might need more computational power and more extensive datasets. The deeper model will be more sensitive to initialization, learning rate, architecture, and regularization settings.

Thus, the deeper architecture does not have to be a better one. The important questions include whether the network's architecture allows representing the problem and generalizing on the new data.

Neural Networks and Deep Learning: Where One Ends and Another Begins

Deep learning is an entirely new approach as compared to neural networks. Deep learning can be perceived as an extension of the traditional approach.

An artificial neuron applies a certain transformation to the input. Multiple neurons are organized into layers. Layers create a network. Networks are trained by adjusting their parameters through optimization.

The difference between the two approaches lies in the scale of the problem.

It is this that laid the groundwork for many of the systems described in later chapters. The abilities of neural networks to handle numerical structures are enhanced significantly when they are tailored for images, language, sound, and other types of data.

Next Chapter

Chapter 07:

When Machines Learn to See and Understand Language

Neural networks provide the foundation for many modern AI systems, but different types of data require different ways of representing and processing information. The next chapter explores how neural networks learn from images and sequences, and introduces convolution, embeddings, attention, and transformers as important ideas behind modern vision and language systems.

Chapter 07

When Machines Learn to See and Understand Language

Neural networks represent a general way of learning patterns in data, but the format of that data can make a difference. An image is organized in a very different way than a sentence, and the structure of sentences is quite unlike the structure of sets of independent numeric features.

Consequently, AI systems have developed special representations of data and mechanisms for working with those different kinds of representation. Models for computer vision need to be able to handle the numeric representations of images and learn spatial patterns. Language requires models that can operate on symbol sequences and learn relationships between words and their contexts.

The invention of neural network architectures made possible the design of models that could specialize for those tasks. Convolutional neural networks were one of the big innovations for visual data, and sequence-based models gave early tools for handling language and other kinds of sequences. More recently, attention and transformer architectures provided a different approach to handling relationships in sequences.

These concepts are an important link between the neural networks covered in the previous chapter and more modern AI systems.

Learning from Images

Images as Numerical Data

A computer does not perceive an image the way a human being does. An image in digital format is described using numerical data.

In a grayscale image, each individual pixel may be assigned a value based on its brightness. Using a simplistic approach, dark pixels will have low numbers while bright pixels will have high numbers.

A grayscale image may therefore be described using a matrix:

General $m \times n$ matrix:

$$X = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1n} \\ x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix}$$

Each element represents the value of one pixel.

Color images require additional information. A common representation uses three channels corresponding to red, green, and blue. The image can then be represented as a three-dimensional array of numerical values.

This representation allows neural networks to process images using mathematical operations.

Pixels and Features

Pixel values represent the underlying numerical data in an image, though it often takes more than just the values of single pixels to characterize an image.

An object in a visual scene depends on the relationship among many pixels. It is formed through edges, shapes, textures, and other higher-level structures in patterns among adjacent regions.

Thus, a good visual representation could be expressed as follows:

Pixels → Edges → Shapes → Objects → Prediction

And it is the task of machine learning to learn such a good representation.

Convolution

Convolution is one of the operations used to process images.

It involves a small matrix known as a filter or kernel moving through various areas of the image, and at every point, there is the combination of values of the filter with values of the corresponding pixels, giving rise to new values.

This can be defined in a simple manner as follows:

$$\text{Output}(i, j) = \sum_{\Sigma_m} \sum_{\Sigma_n} \text{Input}(i + m, j + n) \cdot \text{Kernel}(m, n)$$

This filter can move around the entire image. In this way, the model is able to learn to detect any pattern that exists in various locations.

For example, a filter may be sensitive to a specific direction of an edge, while other filters detect other patterns.

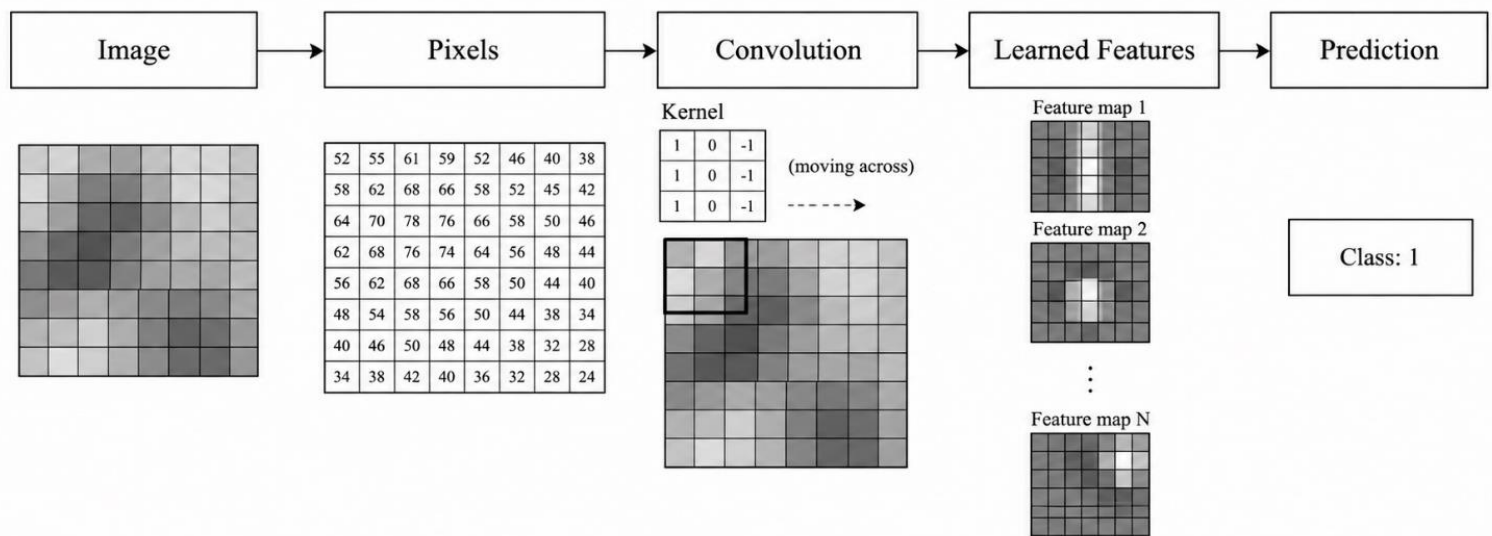
Convolutional Neural Networks

A Convolutional Neural Network, or CNN, uses convolutional operations as an important part of its architecture.

Instead of treating every pixel as an unrelated feature, convolution allows the model to consider local relationships between neighboring pixels. As the information passes through several layers, the network can construct increasingly complex representations.

An early layer may detect simple visual patterns. Later layers can combine these patterns into larger structures, eventually supporting tasks such as image classification or object detection.

CNNs therefore demonstrate an important principle of deep learning: the architecture itself can take advantage of the structure of the data.



Learning from Sequences

Sequential Data

Some data cannot be viewed as independent samples. In some problems, the order of the data is significant.

The data sequence could be written as:

$x_1, x_2, x_3, \dots, x_n$

The interpretation of one data point might depend on other data points preceding or following it.

This applies to sequences like sentences, time series measurements, audio data, and event sequences, among others. The key here is that the model should factor in the dependencies between the positions of these sequences.

This sets sequence processing apart from regular tabular predictions where the model should have a way to represent the data points.

Text as Data

The human language does not have a direct representation of meaning within a computer. In order for a model to process text, the text needs to be transformed into numerical form.

A sentence can first be broken down into smaller entities known as tokens. These tokens can be an entire word, a portion of a word, or any other entity within the text, based on the specific system used.

For instance, a sentence can be represented by:

Text → Tokens → Numerical Representations

Tokens and Representations

The token itself does not carry the complete meaning of a word in all situations. It is thus required that the model should have a representation that captures the relationship between different tokens effectively.

The most common form of representation is called an embedding.

It is a vector of numbers corresponding to a token:

$$\mathbf{x} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n]$$

Tokens which share a similar pattern of usage may develop similar representations.

However, the representation of individual tokens alone is not sufficient. Language is highly contextual in nature.

The meaning of a word may be altered based on the presence of other words in the sentence.

Word-to-word Relationship Learning

Some of the early neural network approaches to natural language processing used a sequential process, where each step was processed independently. This was achieved using architectures such as recurrent neural networks.

This architecture allowed for a sequential process, but when dealing with lengthy sequences, it had one downside. It could be challenging to keep information from an earlier position in a sequence while processing other subsequent ones.

It was not about keeping the words; the problem was that the model would need to recognize what is essential in the sequence at that moment.

This was the rationale behind attention-based approaches.

Attention and Transformers

The Concept of Attention

Attention allows a model to attend to various portions of its input while creating the output representation.

While processing its inputs, the model does not consider all elements equally. Some may have more significance than others in the processing of the particular token.

By focusing on other tokens in the sequence, the model can understand the importance of the particular token to the overall meaning of its representation.

This concept is especially helpful in language since relationships between words could appear far away in a sequence.

Self-Attention

With self-attention, the elements in a sequence will interact with each other within that sequence.

Three matrices are the basic building blocks:

Q = Queries

K = Keys

V = Values

Attention mechanism in its simplest form:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

The role of the query is to show what the position is looking for. Keys show information which can be matched to the query and values have information that can be used for producing the output.

As a result of this operation, the attention weights are determined.

The crucial point is that one token's representation depends on the other tokens in the sequence.

Transformer Architecture

The architecture of Transformers was designed to build sequence models with attention mechanism being the central piece rather than recurrent processing.

Transformer consists of multiple layers where self-attention and transformations take place.

The process can be described in the following simplified manner:

Tokens → Embeddings → Attention → Transformation → Output

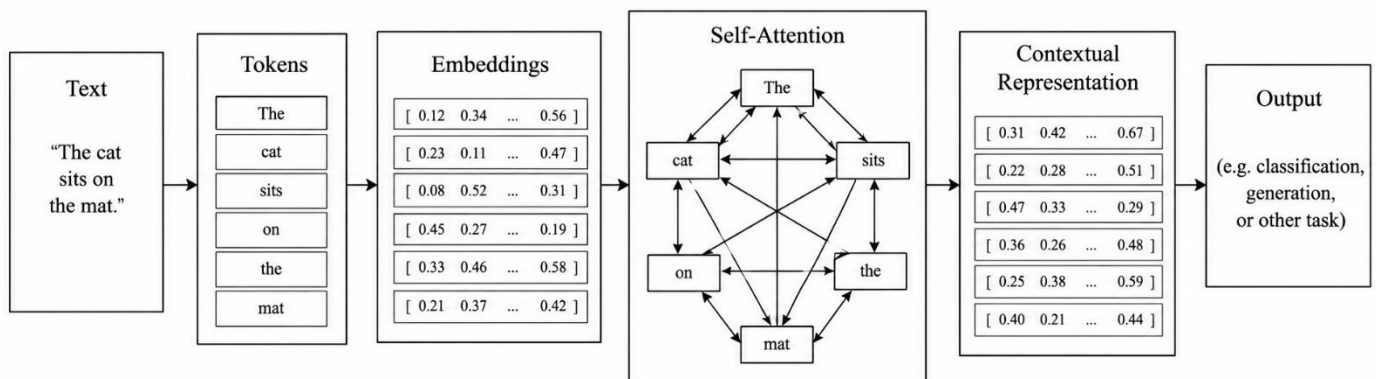
Also, transformers can process many positions of the sequence at once while training. This helped with scaling up the training of large language models.

Recurrent Sequence Models to Transformers

The shift from recurrent sequence models to transformers was crucial in that it affected the way the relationship in long sequences could be represented.

By doing away with the need for information to go through the sequence step by step, attention makes it possible to determine the relationship between different positions.

However, this doesn't necessarily mean that transformers think about languages in the same way as humans. They learn their representations from the data available. The architecture learns the statistical relationship between tokens and structure from training.



Different Types of Data

Vision Models

Models of computer vision are constructed in order to extract useful data from visual information. Depending on the purpose, the model will classify an image, recognize objects, segment regions, or create a representation that can be further used by some other system.

The core idea is similar to that of the neural networks we have already discussed: the numeric input is converted via learned parameters into the useful representation and, ultimately, into the output.

The difference lies only in the architecture that makes use of the inherent structure of visual information.

Language Models

Language models are trained to find the patterns in sequences of tokens. While training, they process vast amounts of text and fine-tune their parameters according to the goal of the particular model.

The language model can learn the statistical connections between words, phrases, syntax, and even broader patterns of usage. Contemporary language models can utilize the learned representations in order to generate texts, classify, summarize, translate, and answer questions.

But the ability to produce coherent language does not necessarily imply correct knowledge about the facts. The model is able to generate fluent output without being actually correct.

Multimodal Learning

Real-life scenarios often use several modalities to represent information. A human can convey his/her ideas via text and images, whereas an artificial intelligence system requires combining visual information with language.

Multimodal learning can be understood as learning with several types of information, including:

Text + Images

Audio + Text

Images + Text + Video

Here it is important to note that processing each modality separately does not serve as the ultimate goal. Multimodal learning implies learning of relationships between different types of information.

For instance, if there is an image and a sentence about this image, then both types provide complementary information. A multimodal learning system, which is able to represent both types of information, can learn relationships between the visual and linguistic representations.

Shared Representations

Another interesting concept of multimodal artificial intelligence is the possibility of transformation of different types of information into compatible representations.

Simpler representation can look like:

Text -> Representation

Image -> Representation

Audio -> Representation

If all these representations reside within the same or connected space, then a model can learn relationships between them.

As a result, various multimodal systems are possible – describing images, answering questions on visual information, interpreting diagrams, and integrating several information sources.

It is important to note that there is no single form of input to which neural networks apply. In other words, the basic principles of representation learning can be used to represent other types of inputs as well.

Next Chapter

Chapter 08:

Modern Artificial Intelligence

Neural networks, deep learning, vision models, language models, and transformers form much of the technical foundation of modern AI. The next chapter brings these ideas into the systems people interact with today, including generative AI, foundation models, multimodal systems, and AI agents. It also examines the capabilities and limitations that become important when these models move from research into practical use.

Chapter 08

Modern Artificial Intelligence

All of the concepts presented within this book are part of an interconnected narrative. There is a way to think about intelligence through a computational problem, data represents the source of information from which to learn patterns, machine learning offers techniques for finding them, and neural networks allow us to express them with increasing power.

Modern artificial intelligence is an expansion of these principles on a large scale.

Contemporary AI systems are capable of dealing with massive amounts of data, learning representations that have billions of parameters, dealing with different types of information, generating novel content, and integrating with other systems. As a result, modern AI enables systems to generate text, interpret images, write code, generate images, answer questions, and help perform various tasks.

Nevertheless, the creation of effective AI models cannot simply be about scaling them up. There is an intricate interplay between data, models, computation, training techniques, evaluation procedures, and human intervention.

This concluding chapter summarizes the key ideas and explores the concepts behind modern AI, including generative AI, foundation models, multimodal systems, AI agents, and their capabilities and limitations.

Generative AI

What is Generative AI?

There are many machine learning algorithms that are used to predict labels, predict numeric values, and detect patterns. In generative AI, the algorithm aims at something different: it generates new data based on the patterns discovered in existing data.

In a generative algorithm, the learned representation of training data is used for generating new outputs.

Depending on the algorithm, the generated output can be:

Text

Image

Audio

Video

Code

Or any other kind of structured or unstructured data

It's important to note that the generated output isn't copied from one of the training examples, but rather generated according to the patterns learned by the algorithm.

Simplified explanation:

Training data → Learning process → Generative algorithm → Generated output

Learning to Generate

In order to generate something useful as an output, the model needs to learn something about relationships within its training data.

For the language model, this can mean learning relationships between tokens and predicting the most probable information to appear next. For the image generation model, it will mean learning representations of visual information and applying them to generate images.

The specific way of training a particular architecture is different but still somehow connected with the idea discussed above – the model learns the parameters from the data and applies them to transform the input into the output.

So generative AI is a subset of machine learning rather than an entirely separate field.

Text Generation

Today's language models are able to generate text in a process that repeats generating predictions based on available context.

The simple algorithm is as follows:

Context → Prediction → New Token → New Context → Prediction

And this can go on until the model generates the complete output.

The quality of the result depends on learned representations of the model, its architecture, data used for training, and the generation process.

Language generation can be used for writing, summarizing, translation, programming, question answering, and many more purposes.

On the other hand, generated text cannot be guaranteed to be accurate as well. It is possible for the model to create an answer that looks grammatically sound yet is incorrect. This is another reason why validation is essential.

Data Image and Other Data Generation

The same concept can be used to apply not just language.

Generative models can learn patterns from image data and generate images based on that. Others will generate sounds, video, molecules, and other types of data.

The difference may be in techniques but the approach is common:

Learn patterns in the data -> Store patterns -> Generate data

It should be noted that in this case the output does not correspond to the stored pattern but rather generated by the learned structure.

Foundation Models

Large Scale Pretraining

Another major innovation in modern artificial intelligence is the introduction of large scale pretraining.

Rather than training a model for a single narrowly defined task, the model could be first trained on a very large set of data. At this stage, the model would learn general representations which then can be utilized in many different tasks.

It can be illustrated as follows:

Large Data Set -> Pretraining -> General Representation -> Application

Pretraining requires computational power and large amounts of data. But the final model becomes a foundation for multiple applications.

Large Language Models

Large Language Models, or LLMs for short, are neural nets that are trained on massive volumes of text and associated data.

Modern LLMs rely on the transformer model architecture. The parameters of such models store the statistics obtained at training time and enable the processing and generation of language by the model.

An LLM does not have a database of phrases that can be returned when it gets a query.

The parameters of the model store the learned relationships that affect the output of the system.

Transfer and Adaptation

A foundation model can be adapted for some particular use case without retraining an entire model from scratch.

Some adaptation methods may include further training, instructing, fine-tuning, accessing external information, etc.

The general logic is:

Foundation Model → Adaptation → Specialized Functionality

In this way, it may become possible to build lots of applications out of one base model.

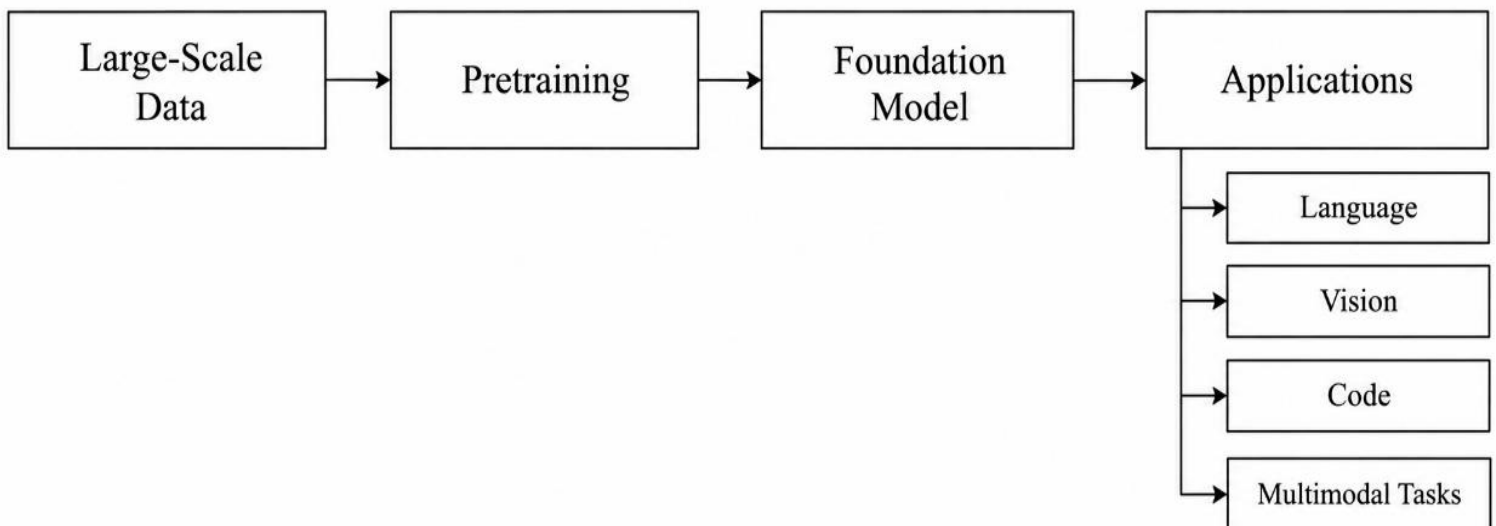
General-Purpose Models

Classical machine learning models are generally built to perform a particular task. The model may be employed to classify images, make price predictions, or recognize a particular kind of signal.

Foundation models have a wider set of potential tasks they can be applied to.

The same model could be used to write, summarize, classify data, reason, assist coding, or interact with other models.

It doesn't imply that the model can be equally effective in all of these tasks. It is dependent on the task, available data, method of evaluation, and architecture.



Combining Text, Images, and Other Types of Information

Interaction of humans with the world is inherently multimodal. We perceive objects, read text, listen to sounds, and use all of these sources of information when making a decision.

Similarly, AI models can be constructed to accept and process multimodal inputs.

The multimodal model can have:

Text + Image Or even: Text + Image + Sound + Video

The model should have representations for all of these types of information and learn relations between them.

This makes some features hard to implement in a unimodal model.

For instance, a model can get an image and a text question at once and give an answer to that question.

Multimodal Models

Multimodal models expand the idea of representation learning to different types of data.

A simple architecture might look like:

Text → Representation

Image → Representation

Audio → Representation

And these representations could be merged and processed using a common model.

But multimodality is more than being able to handle multiple inputs. The system should be able to learn how different representations relate to each other.

One visual feature could correspond to some word. Some sound might contain information about some event depicted in an image. And some text might refer to something that we see in visuals.

That's what multimodal learning tries to learn within one computational system.

Tools and External Information

Internal parameters of the model are not the only source of information for a modern AI system.

We can connect our AI system to external tools which would bring additional functionalities to our system. These could include search engines, calculators, databases, software environments, sensors, and other services.

And it transforms the way our system looks.

Instead of:

Input → Model → Output

we can now use:

Input → Model → Tool → Model → Output

The model has the ability to decide if some outside information or computations are relevant and integrate the new information in the output.

It is important to know that the model and the entire AI system, based on the model, do not necessarily coincide.

AI Agents

An AI agent is a system which uses a model to carry out actions in order to achieve a certain objective.

A simple cycle of an agent is:

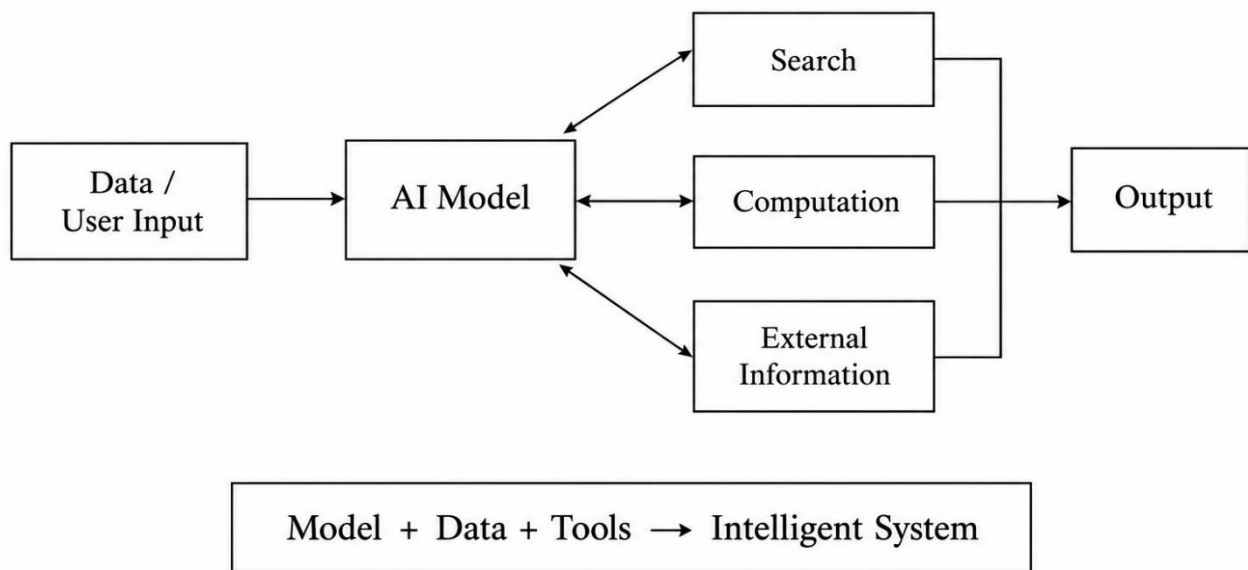
Goal → Observe → Reason → Act → Observe → ...

An agent could obtain information on its surroundings, determine which actions to carry out, utilize some tool, observe what happens and proceed until the work is done or some other criterion is achieved.

This makes an agent distinct from a model which produces only one output.

The model is the source of intelligence in the system while tools, memory, information and controls are additional sources of intelligence.

Thus, AI agents signify a departure from mere models to systems which can be active in multi-step processes.



Intelligence We Build

AI Capacities and Limitations

Current AI systems are capable of performing actions that were previously only possible through considerable human work. Such systems are capable of pattern recognition, content generation, language processing, image analysis, code writing, and even communication with external systems.

Yet, capability should not be equated with intelligence.

A computer program may have tremendous capacity to execute specific tasks but yet fall apart completely when faced with challenges that look straightforward to a person.

The reason for such behavior might lie in misinterpretation of the situation, lack of data for making decisions, providing faulty results, or reacting unexpectedly to data that is

different from training one.

Such limitations are not something apart from how AI works. These are an inherent part of how machine learning algorithms learn.

A model is trained on the basis of certain data and objective. When there is something missing from the former or the latter does not adequately describe the desired behavior, the system will carry those deficiencies.

Reliability and Evaluation

With increasing ability of AI systems comes the importance of evaluation.

One performance measure is usually insufficient to establish reliability. Evaluation can be necessary for looking at the issues of accuracy, robustness, consistency, safety, bias, and performance under different conditions.

For instance, a system may work effectively on a benchmark dataset but prove unable to cope with some slight changes in the input data. There could be another system that produces fluent responses but delivers false information.

The concept of generalization discussed earlier in the book is directly related to this issue.

A good AI system should work not only with the sample data but also with new cases in the operating environment.

Responsibility and Safety in AI

Developing an AI system entails more than making it perform well.

It may affect decision-making, provide information, interact with users, and act via external systems. For this reason, there are concerns about how an AI system can go wrong and what the consequences of such errors might be.

Important questions include:

1. Quality and bias of data
2. Privacy
3. Security
4. Reliability
5. Transparency
6. Human supervision
7. Responsible use

It should be noted that there is no one formula that will help solve all these problems.

They concern the use of data, system design, evaluation, deployment, and human supervision of an AI system.

For this reason, responsible AI will become part of the engineering and science process.

Human Intelligence and Machine Intelligence

The evolution of artificial intelligence leads to the general problem of the connection of human intelligence with machine intelligence.

While machines are able to process much information very quickly and find certain statistical relations which could be hard for people to find, humans give goals, background, values, judgments, and responsibility.

The best way to think about contemporary AI, thus, is not to consider it as the replacement of all kinds of human intelligence, but to see it as a set of systems with their specific abilities and restrictions.

The critical issue is not just the fact of intelligence of some machine. The key thing is what kind of intelligence is possessed by the system, how it was acquired, where it operates successfully, and where it does not.

The Bigger Picture

The evolution of artificial intelligence can now be thought of as an interrelated chain of concepts.

Data gives us information.

Machine learning gives us ways to learn patterns in this information.

Neural networks give us adaptable computational architectures.

Deep learning enables models to learn more and more complicated representations.

Architectures of vision and language adapt these concepts to different types of data.

Transformers and attention enable modeling of relationships in sequences on a large scale.

Generative artificial intelligence makes learned representations applicable to generating new data.

Foundation models make general capabilities available that can be applied to many use cases.

Multimodal models link different types of information.

AI agents combine models with software and other systems in order to perform multistep tasks.

The field evolves because each of these concepts creates new opportunities but also new challenges.

Thus, artificial intelligence is not a single algorithm or a technology. It is a field that is based on mathematics, statistics, computer science, data, optimization, and intelligent behavior research.

These ideas combined together give us the systems of artificial intelligence.

This same principle, which was introduced at the outset of this book, is still central to the discipline of study: Intelligence, in its computational instantiation, must inevitably take form in terms of information, computation, and structure learned.

End of the Book

But learning about how today's AI technologies operate is not the endpoint of researching artificial intelligence.

It is an ever-evolving discipline as researchers develop new models, new architectures, new data sets, and new applications of AI.

With an understanding of the fundamentals, you can take a critical stance toward these developments without regarding the technology as a black box.

A model can get bigger. An architecture can get different. A new method can be used instead of an old one. But the same questions will still arise:

What data is learned by the model?

How is it represented?

How does the model learn?

How do we know it works?

Where does it fail?

And how should we humans use our creations?

This list of questions will accompany the development of artificial intelligence moving forward.

Abdallah Mohamed

Author of A Mind Made of Data

2026